CENTRO PAULA SOUZA
FACULDADE DE TECNOLOGIA DE ARARAQUARA
CURSO DE GRADUAÇÃO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DANIEL VELLUTO BENTO
PEDRO LUIS OTRENTE DE CAMPOS

GASTRA

Projeto de pesquisa apresentado ao Curso de Análise e Desenvolvimento de Sistemas da Faculdade de Tecnologia de Araraquara, como requisito parcial para desenvolvimento do Trabalho de Graduação.

Orientador: Prof. Me. Leonardo José de Lima Ferrucci

Araraquara
6º semestre/2026

LISTA DE ILUSTRAÇÕES

Figura 1 - Business Model Canvas do GASTRA 15
Figura 2 – Diagrama de casos de uso do núcleo de comandas 25
Figura 3 – Diagrama de casos de uso dos blocos analíticos 26
Figura 4 – Diagrama de casos de uso do Cliente 27
Figura 5 – Diagrama Entidade-Relacionamento conceitual do GASTRA 32
Figura 6 – Modelo lógico obtido por engenharia reversa do banco implementado 34
Figura 7 – Diagrama de implantação do GASTRA 37
Figura 8 – Camadas do backend e dependências permitidas 38
Figura 9 – Diagrama de atividade da alocação de garçons 39

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais do GASTRA 21
Quadro 2 – Requisitos não funcionais do GASTRA 23
Quadro 3 – Regras de negócio do GASTRA 23
Quadro 4 – Casos de uso do GASTRA por grupo 24
Quadro 5 – Histórias de usuário, critérios de aceite e testes que os comprovam 27
Quadro 6 – Mapa de dados pessoais e proteção correspondente 35
Quadro 7 – Principais decisões de arquitetura 41

LISTA DE TABELAS

Tabela 1 - Estimativa de custos mensais de infraestrutura 18

LISTA DE ABREVIATURAS E SIGLAS

ABRASEL – Associação Brasileira de Bares e Restaurantes
API – Application Programming Interface
BI – Business Intelligence
CI/CD – Continuous Integration/Continuous Deployment
DER – Diagrama Entidade-Relacionamento
HTML – HyperText Markup Language
HTTP – Hypertext Transfer Protocol
IA – Inteligência Artificial
IP – Internet Protocol
JSON – JavaScript Object Notation

JWT – JSON Web Token

KPI – Key Performance Indicator

LGPD – Lei Geral de Proteção de Dados

MER – Modelo Entidade-Relacionamento

OWASP – Open Worldwide Application Security Project

QR Code – Quick Response Code

RF – Requisito Funcional

RN – Regra de Negócio

RNF – Requisito Não Funcional

SaaS – Software as a Service

SQL – Structured Query Language

SSL – Secure Sockets Layer

TG – Trabalho de Graduação

TOTP – Time-based One-Time Password

UC – Caso de Uso (Use Case)

US – História de Usuário (User Story)

Sumário

1 INTRODUÇÃO 1
1.1 CONTEXTUALIZAÇÃO 1
1.2 PROBLEMATIZAÇÃO 1
1.3 OBJETIVOS 2
1.3.1 Objetivo Geral 2
1.3.2 Objetivos Específicos 2
1.4 JUSTIFICATIVA 3
1.5 METODOLOGIA DA PESQUISA E DESENVOLVIMENTO 4
1.6 ESTRUTURA DO TRABALHO 5
2 REVISÃO BIBLIOGRÁFICA 7
2.1 CONCEITOS E REGRAS DE NEGÓCIO 7
2.2 TECNOLOGIAS E ARQUITETURA 8
2.2.1 Business Intelligence e apoio à decisão 10
2.2.2 Ciência de dados: regras de associação e clusterização 10
2.2.3 Programação linear e o problema de designação 11
2.2.4 Proteção de dados e a LGPD 12
2.3 TRABALHOS CORRELATOS E SIMILARES 12
3 MODELAGEM DE NEGÓCIO E ESCOPO DO PRODUTO 15
3.1 MODELO DE NEGÓCIOS (CANVAS) 15
3.2 VIABILIDADE FINANCEIRA E CUSTOS 17
4 ENGENHARIA DE SOFTWARE E PROJETO DO SISTEMA 20
4.1 LEVANTAMENTO E ESPECIFICAÇÃO DE REQUISITOS 20
4.1.1 Requisitos funcionais 21
4.1.2 Requisitos não funcionais 22
4.1.3 Regras de negócio 23
4.2 MODELAGEM DO SISTEMA (CASOS DE USO / HISTÓRIAS DE USUÁRIO) 24
.3 MODELAGEM DE DADOS (MER / DER) 31
4.3.1 Modelo conceitual 31
4.3.2 Modelos lógico e físico 33
4.3.3 Normalização 34
4.3.4 Proteção de dados na modelagem 35

4.4 ARQUITETURA E PROTOTIPAÇÃO (UI/UX) 36
4.4.1 Visão geral e implantação 36
4.4.2 Backend em camadas 37
4.4.3 Camada analítica 39
4.4.4 Decisões de arquitetura 41
5 DESENVOLVIMENTO, RESULTADOS E TESTES 42
6 CONSIDERAÇÕES FINAIS 43
REFERÊNCIAS 44

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO

Restaurantes produzem diariamente um grande volume de dados operacionais — vendas, ocupação de mesas, horários de atendimento e pedidos —, mas grande parte das decisões gerenciais nesse setor ainda é tomada de forma intuitiva. O peso desse mercado reforça a relevância do problema: de acordo com a Associação Brasileira de Bares e Restaurantes, a alimentação fora do lar movimentou R$ 495 bilhões em 2025, R$ 40 bilhões a mais que em 2024 (ABRASEL, 2025a). Em uma operação de escala tão grande, pequenas ineficiências na gestão, quando multiplicadas pelo volume de atendimentos, resultam em impacto financeiro significativo. Apesar do grande volume de dados já gerado na rotina de um restaurante, técnicas de Business Intelligence (BI), ciência de dados e otimização matemática ainda são pouco exploradas como apoio à gestão na maioria dos estabelecimentos — sobretudo os de menor porte, que normalmente não dispõem de equipe ou ferramentas dedicadas à análise desses dados.

1.2 PROBLEMATIZAÇÃO

Apesar do grande volume de dados operacionais gerados diariamente em um restaurante, falta às equipes de salão e gestão uma ferramenta que transforme esses dados em apoio direto à decisão. Essa lacuna aparece de forma concreta na distribuição de garçons entre as praças do salão: hoje esse processo costuma ser hierárquico e informal — o garçom mais antigo escolhe a praça que prefere, e o restante é distribuído por sorteio ou pelo critério do metre —, sem considerar o potencial de faturamento de cada área. Como os garçons costumam receber por comissão sobre as vendas, essa distribuição sem critério objetivo gera percepção de injustiça entre a equipe, já que praças com maior movimento resultam em ganhos maiores independentemente do desempenho individual do garçom. Da mesma forma, a recomendação de pratos ao cliente depende hoje inteiramente da experiência pessoal de cada garçom, sem apoio de dados sobre padrões de consumo — o que resulta em sugestões limitadas, sobretudo para clientes novos na casa.

Diante desse cenário, este projeto busca responder: como apoiar a tomada de decisão em um restaurante a partir dos próprios dados gerados pela operação? De forma mais específica, busca-se responder às seguintes questões: como distribuir os garçons de maneira mais equilibrada entre as praças do salão, com base no potencial de faturamento de cada uma? Como

identificar padrões de consumo dos clientes para apoiar a recomendação de combinações de pratos? E como realizar essas análises respeitando os limites impostos pela Lei Geral de Proteção de Dados?

1.3 OBJETIVOS

1.3.1 Objetivo Geral

Desenvolver o GASTRA (Gestão Analítica de Restaurantes), um sistema de apoio à decisão para a gestão de restaurantes à la carte, que combine relatórios de Business Intelligence, recomendação de pratos por ciência de dados e distribuição de garçons entre praças por programação linear, com um módulo operacional de registro de pedidos que alimenta esses três blocos analíticos com dados reais da operação, em conformidade com a Lei Geral de Proteção de Dados.

1.3.2 Objetivos Específicos

Levantar os requisitos do sistema por meio de entrevista qualitativa e questionários aplicados a garçons e clientes, formalizando-os em requisitos funcionais, não funcionais e regras de negócio;

Modelar e implementar um banco de dados relacional para armazenar as informações operacionais de comandas, cardápio, praças, alocação de garçons e auditoria;

Desenvolver um módulo operacional de registro de pedidos (abertura e fechamento de comanda, lançamento de itens), responsável por alimentar os blocos analíticos com dados reais da operação;

Desenvolver a aplicação web — backend em ASP.NET Core e frontend em Angular —, incluindo os relatórios de Business Intelligence para visualização de indicadores por praça, por garçom e por horário;

Aplicar técnicas de ciência de dados (clusterização e regras de associação) para identificar padrões de consumo e recomendar combinações de pratos ao garçom;

Modelar e resolver, por meio de programação linear, o problema de distribuição de garçons entre as praças do salão, considerando o potencial de faturamento de cada praça;

Garantir a conformidade do sistema com a Lei Geral de Proteção de Dados, com minimização de dados e controle de acesso por papel de usuário;

Validar o sistema desenvolvido por meio de testes automatizados e cenários de uso.

1.4 JUSTIFICATIVA

O desenvolvimento do GASTRA exige a integração de competências de diferentes áreas da formação em Análise e Desenvolvimento de Sistemas. O projeto articula conhecimentos de disciplinas como Banco de Dados, para a modelagem das tabelas de comandas, cardápio e alocação; Estatística Aplicada, para a análise de dados e a clusterização de padrões de consumo; Programação Linear e Aplicações, para a distribuição de garçons entre as praças; Engenharia de Software e Programação Web, para a construção da aplicação em camadas; e Segurança da Informação e Ética e Responsabilidade Profissional, para o tratamento de dados sob a LGPD. Dessa forma, o projeto reúne em um único trabalho conhecimentos normalmente desenvolvidos de forma isolada ao longo do curso.

Do ponto de vista técnico, a construção do sistema adota práticas de engenharia de software que vão além do escopo de um exercício de disciplina isolada, como arquitetura em camadas com regra de dependência verificada por teste automatizado, inversão de dependência entre domínio e infraestrutura, e integração entre dois ecossistemas de tecnologia distintos — o backend em C#/.NET e o serviço analítico em Python — comunicando-se por HTTP. Essa arquitetura permite isolar os algoritmos de ciência de dados e programação linear em um serviço dedicado, aproveitando bibliotecas maduras do ecossistema Python (mlxtend, scikit-learn, PuLP) sem comprometer a robustez transacional do núcleo de comandas, que permanece disponível mesmo que o serviço analítico esteja indisponível.

Na prática, o GASTRA contribui para que as decisões gerenciais deixem de se apoiar apenas na intuição e passem a se fundamentar em dados concretos da operação. Esse ganho se estende diretamente aos garçons: como esses profissionais costumam receber por comissão, a praça em que atuam influencia diretamente seus rendimentos, e a entrevista realizada com um garçom de restaurante à la carte confirmou que a distribuição hoje é percebida como desigual, por depender de hierarquia informal entre a equipe. Ao considerar o potencial de faturamento de cada praça, o sistema busca uma alocação mais justa. Do lado do cliente, a recomendação de combinações de pratos e a sinalização de restrições alimentares no cardápio tendem a agilizar o atendimento e reduzir a dependência da experiência individual de cada garçom. Por envolver dados tanto de clientes quanto de funcionários, o tratamento responsável dessas informações é parte central dessa contribuição social: o sistema prioriza análises agregadas, adota a anonimização de dados sempre que possível e limita a coleta ao estritamente necessário, em conformidade com o princípio da necessidade previsto na Lei Geral de Proteção de Dados (BRASIL, 2018).

No campo mercadológico, o setor de alimentação fora do lar é um dos maiores empregadores do país: segundo dados da PNAD Contínua divulgados pela Abrasel, o segmento ocupava 5,56 milhões de pessoas no primeiro trimestre de 2025, com rendimento médio de R$ 2.227 — recorde histórico do setor (ABRASEL, 2025b). Por trás desses números está um contingente expressivo de trabalhadores cuja remuneração depende, em boa parte, do movimento e da operação diária dos estabelecimentos, o que reforça a relevância de ferramentas de gestão mais bem fundamentadas em dados, sobretudo para restaurantes de médio porte que hoje não têm acesso a esse tipo de solução.

1.5 METODOLOGIA DA PESQUISA E DESENVOLVIMENTO

Quanto à natureza, esta pesquisa caracteriza-se como aplicada, com abordagem de desenvolvimento tecnológico, uma vez que tem como resultado a construção de um sistema de software voltado a um problema concreto de gestão. Quanto aos procedimentos, combina pesquisa bibliográfica — para a fundamentação teórica dos temas envolvidos — e estudo de caso, conduzido junto a um restaurante colaborador à la carte, por meio de entrevista qualitativa semiestruturada com um garçom e de questionários aplicados a garçons e clientes.

O levantamento de requisitos partiu de duas frentes complementares. A primeira, quantitativa, consistiu em dois questionários aplicados pela ferramenta Jotform: um dirigido a garçons/atendentes (n = 2 respondentes), cobrindo fluxo operacional, alocação de praças, cardápio e gamificação; e outro dirigido a clientes de restaurantes à la carte (n = 13 respondentes), cobrindo experiência de atendimento, identificação de restrições alimentares e percepção sobre personalização e uso de dados. A segunda frente, qualitativa, consistiu em uma entrevista semiestruturada com um garçom de restaurante à la carte, conduzida com roteiro próprio e gravação transcrita exclusivamente para extração de trechos relevantes, sendo o áudio descartado após a transcrição. As duas frentes foram cruzadas para identificar convergências e aprofundar achados que a amostra pequena do questionário de garçom, isoladamente, não sustentaria — a exemplo da percepção de injustiça na distribuição de praças, confirmada tanto pelo questionário quanto pela entrevista.

O desenvolvimento seguiu processo iterativo baseado no Scrum (SCHWABER; SUTHERLAND, 2020), adaptado à realidade de uma dupla, sem os papéis formais de Product Owner e Scrum Master: as tarefas foram registradas como issues, com critérios de aceite, prioridade e marco, e organizadas em sprints semanais no quadro do GitHub Projects, com revisão e retrospectiva ao final de cada sprint e planejamento da seguinte a partir delas. O controle de versão seguiu o modelo GitFlow (DRIESSEN, 2010), com as branches main, dev e

uma branch por funcionalidade. Toda mudança entrou por Pull Request e só foi integrada depois da revisão do outro integrante, regra imposta pelo próprio repositório; a cada Pull Request, a integração contínua do GitHub Actions (GITHUB, 2026) compilou o sistema e executou os testes automatizados, com o resultado exibido ao revisor antes da decisão. As convenções desse processo estão registradas no guia de contribuição do repositório.

A modelagem e a implementação do banco de dados relacional seguiram a abordagem Code First, com o esquema físico gerado a partir de migrations do Entity Framework Core e validado por engenharia reversa contra o modelo conceitual. A aplicação web foi desenvolvida com backend em ASP.NET Core, organizado em arquitetura de camadas com regra de dependência verificada por teste automatizado, e frontend em Angular. As telas foram desenhadas primeiro em um protótipo navegável, e as telas implementadas passaram por avaliação heurística (NIELSEN, 1994), descrita na seção 5.2.5. A camada analítica, em Python, expõe os algoritmos de recomendação de pratos, de segmentação das comandas em perfis de consumo e de alocação de garçons por meio de um serviço HTTP consumido pelo backend.

Em razão das restrições impostas pela Lei Geral de Proteção de Dados ao uso de dados pessoais reais, e da ausência de histórico de pedidos do restaurante colaborador, os algoritmos de recomendação e de alocação foram validados com dados simulados, gerados de forma determinística e com premissas explícitas, descritas junto a cada análise. Essa escolha permite avaliar o comportamento dos módulos analíticos sem expor dados de clientes ou funcionários reais; a arquitetura do sistema, no entanto, já contempla a leitura de histórico real assim que ele estiver disponível, com o dado simulado como alternativa apenas quando o volume de dado real for insuficiente.

A validação do sistema ocorreu em três frentes: testes automatizados de unidade, integração e arquitetura, cobrindo os quatro blocos do projeto (núcleo de comandas, alocação por programação linear, recomendação de pratos e conformidade com a LGPD); a avaliação heurística das telas; e cenários operacionais de ponta a ponta, executados com o sistema completo em contêineres. Os algoritmos analíticos foram avaliados, além disso, por experimentos sobre os dados simulados, cujas métricas — a taxa de acerto da recomendação, a silhueta da segmentação e a desigualdade de faturamento entre os garçons — são apresentadas no capítulo 5.

1.6 ESTRUTURA DO TRABALHO

Este trabalho está organizado em seis capítulos, além dos elementos pré e pós-textuais exigidos pela norma ABNT. O primeiro capítulo, Introdução, contextualiza o setor de

restaurantes e a problemática que motivou o desenvolvimento do GASTRA, apresentando os objetivos geral e específicos do trabalho, a justificativa sob as perspectivas acadêmica, técnica, social e mercadológica, e a metodologia de pesquisa e desenvolvimento adotada. O segundo capítulo, Revisão Bibliográfica, apresenta os conceitos e regras de negócio do domínio de restaurantes, a fundamentação teórica das tecnologias, da arquitetura e de cada bloco do sistema — Business Intelligence, ciência de dados, programação linear e proteção de dados — e a comparação do GASTRA com trabalhos correlatos. O terceiro capítulo, Modelagem de Negócio e Escopo do Produto, apresenta o modelo de negócios do sistema por meio do Business Model Canvas e a estimativa de recursos e custos de infraestrutura necessários para colocá-lo em produção. O quarto capítulo, Engenharia de Software e Projeto do Sistema, detalha os artefatos técnicos do projeto: o levantamento e a especificação dos requisitos funcionais e não funcionais, a modelagem do sistema por casos de uso, a modelagem de dados por meio do Modelo de Entidade-Relacionamento e do Diagrama Entidade-Relacionamento, e a arquitetura da solução. O quinto capítulo, Desenvolvimento, Resultados e Testes, apresenta o produto final construído, com as telas do sistema em funcionamento, e os testes e a validação do software. O sexto e último capítulo, Considerações Finais, sintetiza as conclusões do trabalho, avalia o cumprimento dos objetivos propostos e discute as dificuldades encontradas e as possibilidades de trabalhos futuros. Ao final, apresenta-se o manual do usuário como apêndice.

2 REVISÃO BIBLIOGRÁFICA

2.1 CONCEITOS E REGRAS DE NEGÓCIO

O GASTRA atua no domínio operacional de restaurantes à la carte com atendimento por garçom, no qual alguns conceitos de negócio são centrais para a compreensão do sistema.

O salão do restaurante é dividido em praças, áreas que agrupam um conjunto de mesas e que possuem, cada uma, uma quantidade determinada de vagas para garçons por turno. A distribuição dos garçons entre as praças costuma seguir, na prática observada, critérios hierárquicos e informais — o garçom mais antigo escolhe a praça de sua preferência, e as demais são distribuídas por sorteio ou pelo critério do metre —, sem levar em conta o potencial de faturamento de cada área. Esse critério é relevante porque, no modelo de remuneração predominante no setor, o garçom recebe parte de sua renda por comissão sobre o valor vendido nas mesas que atende; praças com maior movimento tendem a gerar comissões maiores, independentemente do desempenho individual do profissional.

O atendimento de cada mesa é registrado em uma comanda, aberta pelo garçom no momento em que o cliente é recepcionado e fechada ao final do atendimento, quando o valor total — incluindo, tipicamente, uma taxa de serviço de 10% sobre o consumo, removível a pedido do cliente — é apurado. Cada item pedido pelo cliente é lançado na comanda com o preço vigente no momento do pedido, preço esse que permanece fixo mesmo que o cardápio seja atualizado posteriormente.

O cardápio de um restaurante à la carte tipicamente apresenta poucas opções voltadas a restrições alimentares específicas — como pratos veganos, vegetarianos, sem glúten ou sem lactose —, o que dificulta tanto a identificação dessas opções pelo cliente quanto a recomendação assertiva pelo garçom. A sugestão de pratos complementares (entrada, bebida, sobremesa) ao cliente é hoje uma habilidade dependente da experiência individual de cada garçom, sem apoio de dado sobre padrões reais de consumo.

Esses conceitos — praça, comissão, comanda e cardápio — formam a base sobre a qual o GASTRA estrutura seus quatro blocos analíticos: a distribuição de garçons entre praças por programação linear, a recomendação de pratos por ciência de dados, os relatórios de Business Intelligence e a conformidade com a Lei Geral de Proteção de Dados no tratamento desses dados operacionais.

2.2 TECNOLOGIAS E ARQUITETURA

A arquitetura do GASTRA é organizada em camadas, um padrão amplamente adotado em engenharia de software para separar responsabilidades distintas de um sistema — apresentação, regras de negócio, contratos de comunicação e acesso a dados — em módulos com baixo acoplamento entre si (PRESSMAN; MAXIM, 2011). No GASTRA, essa separação é levada ao nível de projetos distintos na solução: o domínio (Gastra.Domain) concentra as entidades e as regras de negócio sem depender de nenhuma outra camada, enquanto a infraestrutura (Gastra.Infrastructure) implementa o acesso ao banco de dados e a comunicação com serviços externos a partir de interfaces definidas pelo próprio domínio. Essa inversão de dependência — em que o módulo de mais alto nível define os contratos, e o módulo de mais baixo nível os implementa — permite substituir a tecnologia de persistência ou o algoritmo de recomendação sem alterar a regra de negócio, e possibilita testar o domínio isoladamente, sem a necessidade de um banco de dados real (SOMMERVILLE, 2019). A regra de dependência entre camadas é verificada automaticamente por um teste de arquitetura, que garante que a estrutura definida em projeto não seja violada ao longo do desenvolvimento.

O backend foi desenvolvido em ASP.NET Core (MICROSOFT, 2026a), framework definido no projeto de pesquisa formal do GASTRA para a construção da API responsável pelas regras de negócio, autenticação e gravação no banco de dados. O acesso a dados utiliza o Entity Framework Core (MICROSOFT, 2026b) como mapeador objeto-relacional, com a abordagem Code First, na qual o esquema do banco de dados é gerado a partir das classes do domínio por meio de migrations versionadas — em vez de escrito manualmente em SQL —, o que mantém o histórico de alterações do banco de dados rastreável junto ao histórico de código da aplicação.

O frontend foi desenvolvido em Angular (GOOGLE, 2026), também definido no projeto de pesquisa formal, como aplicação de página única (Single-Page Application) organizada por módulo funcional, de modo que cada grupo de telas (comandas, alocação, análises, cliente) possa evoluir de forma independente, sem interferir nos demais.

A camada analítica — responsável pelos algoritmos de recomendação de pratos e de alocação de garçons — foi implementada separadamente, em Python, e exposta ao backend por meio de um serviço HTTP com FastAPI (RAMÍREZ, 2026). Essa separação segue o princípio de que cada tecnologia deve ser usada onde apresenta a maior vantagem: o ecossistema Python concentra bibliotecas maduras de ciência de dados e otimização matemática — mlxtend (RASCHKA, 2018), scikit-learn (PEDREGOSA et al., 2011) e PuLP (MITCHELL; O'SULLIVAN; DUNNING, 2011) —, que não têm equivalente igualmente consolidado no

ecossistema .NET. Um efeito dessa separação, tratado como decisão de arquitetura, é a independência do núcleo de comandas em relação ao serviço analítico: caso o serviço em Python fique indisponível, apenas a sugestão de pratos e a sugestão de alocação deixam de estar disponíveis temporariamente, enquanto o restante do sistema — abertura de mesa, lançamento de pedido e fechamento de comanda — continua operando normalmente.

O banco de dados relacional utiliza MySQL (ORACLE, 2026), também definido no projeto de pesquisa formal, sobre o qual foram criadas views para consultas agregadas usadas tanto pelos relatórios de Business Intelligence quanto pelo serviço de recomendação de pratos, e triggers para proteger a tabela de auditoria contra alteração e exclusão indevidas. Optou-se por não utilizar stored procedures para regras de negócio: regras como o fechamento de comanda e o cálculo da taxa de serviço já existem no domínio da aplicação e são cobertas por testes automatizados que não dependem de um banco de dados ativo; replicá-las em uma procedure faria a mesma regra existir em dois lugares distintos, com risco de divergência entre eles.

Os quatro componentes — frontend, backend, serviço analítico e banco de dados — são executados em contêineres Docker (DOCKER, 2026). Um contêiner empacota o componente com tudo de que ele precisa para rodar, de modo que a mesma configuração vale no desenvolvimento, na integração contínua e na entrega, e o sistema completo sobe com um único comando em qualquer máquina com o Docker instalado, sem instalar .NET, Node.js ou Python.

A autenticação utiliza o JSON Web Token (JWT), padrão aberto para transmitir, de forma assinada, a identidade e o papel do usuário entre o navegador e o servidor (JONES; BRADLEY; SAKIMURA, 2015). Como um JWT continua válido até expirar, o GASTRA associa a cada token uma chave de sessão guardada no banco, o que permite invalidá-lo no logoff, na troca de papel e na inativação da conta; para os perfis de gestão, o login exige ainda um segundo fator, o código de um aplicativo autenticador. A revisão de segurança tomou como referência o OWASP Top 10 (OWASP FOUNDATION, 2021), lista dos riscos mais críticos de segurança em aplicações web: para cada risco aplicável, a documentação de arquitetura do repositório registra como o sistema o trata.

O GASTRA apoia-se em quatro eixos teóricos, um para cada bloco do sistema: o Business Intelligence, base dos relatórios gerenciais; a ciência de dados, com as regras de associação e a clusterização que sustentam a recomendação de pratos; a programação linear, aplicada à distribuição dos garçons entre as praças; e a Lei Geral de Proteção de Dados, que

orienta o tratamento das informações em todos os outros. As subseções a seguir apresentam os fundamentos de cada eixo e a sua relação com a solução.

2.2.1 Business Intelligence e apoio à decisão

O Business Intelligence (BI) é o conjunto de técnicas e ferramentas voltadas à coleta, organização, análise e apresentação de informações que subsidiam a tomada de decisão gerencial. Segundo Santos e Gibertoni (2022), o BI permite que gestores tomem decisões mais assertivas e ágeis, convertendo grandes volumes de dados em relatórios e indicadores relevantes para a competitividade das empresas. No GASTRA, essa conversão é feita sobre os dados que o próprio sistema registra na operação — comandas, itens pedidos, praças e turnos —, sem planilhas paralelas.

A aplicação do BI ao setor de alimentação encontra respaldo em Martínez Ortiz (2025), que analisa as condições de implementação de Business Intelligence em micro e pequenas empresas do ramo de restaurantes e destaca que o uso de indicadores-chave de desempenho (KPIs) alinhados aos objetivos do estabelecimento contribui para a gestão, a análise de vendas e a experiência do cliente. O GASTRA segue essa orientação: cada indicador apresentado ao Gerente — como o faturamento por praça e por período, os itens mais vendidos e o índice de desempenho dos garçons — foi definido junto com a pergunta de gestão que ele responde e o critério com que é calculado, registrados na documentação de requisitos do projeto.

2.2.2 Ciência de dados: regras de associação e clusterização

A ciência de dados amplia as possibilidades do BI ao aplicar técnicas de mineração sobre os dados operacionais. No contexto de restaurantes, Huallpa Tapia et al. (2019) aplicam mineração de dados e técnicas de classificação para compreender e elevar a satisfação dos clientes, com precisão preditiva superior a 80%, o que evidencia o potencial da análise do comportamento de consumo para apoiar decisões no setor.

A recomendação de pratos do GASTRA apoia-se nas regras de associação, técnica também conhecida como análise de cesta de compras. O algoritmo Apriori (AGRAWAL; SRIKANT, 1994) encontra, em um conjunto de transações, os grupos de itens que aparecem juntos com frequência e, a partir deles, regras do tipo "quem leva A também leva B". Cada regra é avaliada por três medidas: o suporte, fração das transações que contêm A e B juntos; a confiança, fração das transações com A que também contêm B; e o lift, razão entre a confiança e a frequência de B, que indica se levar A aumenta (lift acima de 1) ou diminui (abaixo de 1) a

chance de levar B. Gonçalves e Soler (2012) aplicam essa técnica à cesta de compras do varejo, identificando produtos adquiridos em conjunto para orientar ações comerciais. No GASTRA, a transação é a comanda, e a regra é usada no momento do pedido, para sugerir ao garçom o complemento que costuma acompanhar o que a mesa já pediu.

A clusterização, por sua vez, agrupa elementos semelhantes sem que os grupos sejam conhecidos de antemão. O algoritmo K-Means (MACQUEEN, 1967) parte de uma quantidade k de grupos, associa cada elemento ao centro mais próximo e recalcula os centros até que a divisão se estabilize. Como k precisa ser informado, a escolha costuma usar o coeficiente de silhueta (ROUSSEEUW, 1987), que compara, para cada elemento, a distância média aos elementos do próprio grupo com a distância média aos do grupo vizinho mais próximo: varia de -1 a 1, e valores mais altos indicam grupos mais coesos e mais separados entre si. Cavique (2003) propõe a micro-segmentação de clientes a partir de dados de consumo, combinando a segmentação com a análise da cesta de compras para identificar o produto adequado a cada perfil, e Syed Marzuki, Hall e Ballantine (2014) empregam análise de clusters no próprio setor de restaurantes, para agrupar perfis de gestores a partir de questionários.

O GASTRA combina as duas técnicas, como propõe Cavique (2003), com uma diferença deliberada: os grupos são de comandas, e não de clientes. Como o cliente não é identificado (seção 2.2.4), o perfil é definido apenas pelo que foi pedido na mesa, e a recomendação passa a considerar, além das regras gerais, as regras próprias do perfil de consumo a que a comanda em andamento se assemelha.

2.2.3 Programação linear e o problema de designação

A programação linear trata da alocação de recursos limitados entre atividades concorrentes, otimizando uma função objetivo linear sujeita a restrições também lineares (HILLIER; LIEBERMAN, 2006). Um de seus casos particulares é o problema de designação, em que cada agente deve ser atribuído a uma tarefa de modo a minimizar o custo total. Por ser também um caso particular do problema de transporte, a sua solução ótima é inteira sem que seja preciso impor essa condição ao modelo (HILLIER; LIEBERMAN, 2006), o que permite resolvê-lo com rapidez a cada turno.

A distribuição de garçons entre as praças configura-se como um problema desse tipo. Zaniol (2011) modela a alocação de funcionários como um emparelhamento em grafos bipartidos, resolvido por programação linear, buscando a alocação ideal segundo as aptidões dos trabalhadores. Coelho (2021) desenvolve um modelo de programação linear inteira mista para a designação de entregadores em uma empresa de food delivery, validado com casos reais.

De forma mais próxima a este trabalho, Vieira et al. (2015) determinam escalas de trabalho de funcionários de restaurantes considerando as suas preferências, por programação linear inteira, com o objetivo de homogeneizar a carga de trabalho. O GASTRA compartilha com este último o domínio e o princípio de distribuir de forma mais justa; o critério, porém, não é a carga de trabalho nem a aptidão, mas a oportunidade de faturamento: o custo de cada designação combina a diferença de faturamento entre os garçons e o tempo desde a última praça de alto potencial.

Para verificar se a distribuição fica de fato mais justa ao longo do tempo, é preciso medir a desigualdade. O coeficiente de Gini (GINI, 1912), medida clássica de concentração, vale zero quando todos os valores são iguais e se aproxima de um quando um único elemento concentra o total. No capítulo 5, ele é aplicado ao faturamento médio por turno dos garçons, para comparar a regra do sistema com a distribuição sem critério.

2.2.4 Proteção de dados e a LGPD

Por tratar dados de clientes e funcionários, o GASTRA deve observar a Lei Geral de Proteção de Dados Pessoais (LGPD), Lei nº 13.709/2018, cujo princípio da necessidade estabelece a:

> limitação do tratamento ao mínimo necessário para a realização de suas finalidades, com abrangência dos dados pertinentes, proporcionais e não excessivos em relação às finalidades do tratamento de dados (BRASIL, 2018, art. 6º, inciso III).

Rapôso et al. (2019), em revisão sistemática sobre a LGPD aplicada à tecnologia da informação, apontam os desafios de adaptação dos sistemas às exigências legais e a importância de tratá-las desde o desenvolvimento do software, e não depois dele. O GASTRA seguiu esse caminho: a proteção de dados foi tratada como requisito de projeto. O cliente não é identificado, porque nenhuma funcionalidade precisa saber quem ele é; a restrição alimentar, que pode revelar dado referente à saúde — categoria que a lei classifica como dado pessoal sensível (BRASIL, 2018, art. 5º, inciso II) —, serve ao atendimento da mesa, não chega aos relatórios e é apagada no fechamento da comanda; e as análises usam somente dados agregados. As decisões e os testes que as verificam são apresentados nos capítulos 4 e 5.

2.3 TRABALHOS CORRELATOS E SIMILARES

Para posicionar o GASTRA frente ao que já existe, comparam-se dois trabalhos do mesmo domínio: um sistema comercial de gestão de restaurantes amplamente adotado no

mercado brasileiro e um trabalho acadêmico com escopo mais próximo do núcleo operacional do GASTRA.

O Saipos (SAIPOS, [20--]) é um sistema de gestão para restaurantes em nuvem, utilizado por mais de 20 mil estabelecimentos no Brasil, que centraliza pedidos de múltiplos aplicativos de delivery em uma única tela, emite nota fiscal automaticamente, controla estoque por ficha técnica e oferece relatórios financeiros. O sistema também conta com controle de mesas e comandas operado pelo próprio garçom em tablet, integrado ao restante da gestão, tornando o fechamento de conta mais ágil. Apesar da abrangência operacional e financeira, o Saipos não apresenta, em sua oferta pública, um mecanismo de distribuição de garçons entre praças do salão baseado em critério de equilíbrio de faturamento, nem recomendação de pratos derivada de padrões de consumo por técnicas de ciência de dados — suas funcionalidades de IA concentram-se em automações de estoque e atendimento via chatbot, não em otimização matemática da alocação de equipe. O GASTRA se diferencia justamente nesse ponto: ao invés de um sistema de gestão de amplo espectro (fiscal, estoque, delivery), concentra-se em um recorte mais estreito e analiticamente mais profundo — Business Intelligence, recomendação por clusterização e regras de associação, e alocação de garçons por programação linear —, com a conformidade à LGPD tratada como requisito central de desenho, e não como funcionalidade adicional.

Já o trabalho de Moreira (2018), desenvolvido na Universidade de Brasília, propôs o FoodView, uma funcionalidade de acompanhamento de consumo integrada a um software de restaurante já existente (ClickFood), que permite ao cliente, via leitura de QR Code em sua mesa, visualizar os itens já pedidos, o status de produção de cada item e o valor parcial da conta, sem a necessidade de login. O trabalho tem em comum com o GASTRA a coleta de requisitos por entrevista semiestruturada com usuários reais do restaurante estudado e o uso de metodologia ágil (Scrum) para o desenvolvimento. A diferença central está no escopo: o FoodView entrega uma única funcionalidade voltada ao cliente, sem qualquer componente analítico ou de apoio à decisão gerencial — o próprio autor identifica, como trabalho futuro não implementado, o recebimento de sugestões de produtos ao cliente, funcionalidade que o GASTRA endereça de forma mais ampla e voltada ao garçom, por meio da recomendação baseada em regras de associação (RF09). O GASTRA também vai além do escopo do FoodView ao tratar a distribuição de equipe entre praças como problema central de pesquisa, e não apenas como consulta de informação.

Em comum, os dois trabalhos correlatos confirmam uma tendência já identificada na justificativa deste trabalho: o setor de restaurantes tem investido em tecnologia voltada à

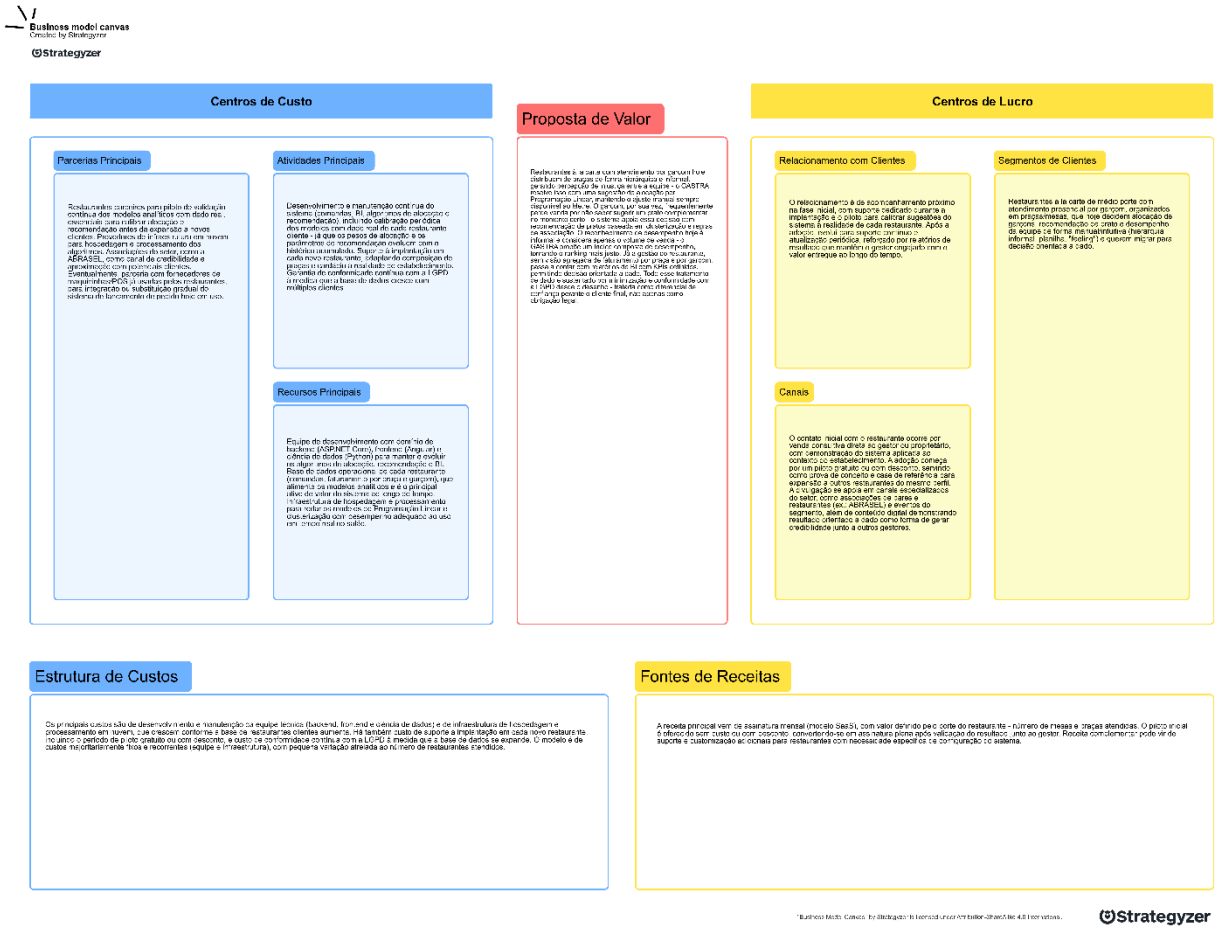
experiência do cliente e à automação operacional, mas a aplicação de técnicas de ciência de dados e otimização matemática ao apoio à decisão gerencial — sobretudo à distribuição de trabalho entre a equipe de forma mensurável e justa — permanece pouco explorada tanto no mercado quanto na produção acadêmica consultada, reforçando a contribuição do GASTRA nesse recorte específico.

3 MODELAGEM DE NEGÓCIO E ESCOPO DO PRODUTO

3.1 MODELO DE NEGÓCIOS (CANVAS)

O modelo de negócios do GASTRA foi estruturado por meio do Business Model Canvas, apresentado na Figura 1, e detalhado a seguir em seus nove blocos constitutivos.

Figura 1 - Business Model Canvas do GASTRA



Fonte: elaborado pelos autores com base no template de Strategyzer (2026).

Proposta de Valor: restaurantes à la carte com atendimento por garçom hoje distribuem as praças do salão de forma hierárquica e informal, o que gera percepção de injustiça entre a equipe — o GASTRA propõe uma sugestão de alocação por programação linear, mantendo o ajuste manual sempre disponível ao metre. O garçom, por sua vez, frequentemente perde venda por não saber sugerir um prato complementar no momento certo — o sistema apoia essa decisão com recomendação de pratos baseada em clusterização e regras de associação. O reconhecimento de desempenho hoje é informal e considera apenas o volume de venda; o GASTRA propõe um índice composto de desempenho, tornando o ranking mais justo. Já a

gestão do restaurante, sem visão agregada de faturamento por praça e por garçom, passa a contar com relatórios de Business Intelligence com KPIs definidos, permitindo decisão orientada a dado. Todo esse tratamento de dado é sustentado por minimização e conformidade com a LGPD desde o desenho, tratada como diferencial de confiança perante o cliente final, e não apenas como obrigação legal.

Segmentos de Clientes: restaurantes à la carte de médio porte com atendimento presencial por garçom, organizados em praças/mesas, que hoje decidem alocação de garçons, recomendação de prato e desempenho da equipe de forma manual/intuitiva (hierarquia informal, planilha, "feeling") e querem migrar para decisão orientada a dado.

Canais: o contato inicial com o restaurante ocorre por venda consultiva direta ao gestor ou proprietário, com demonstração do sistema aplicada ao contexto do estabelecimento. A adoção começa por um piloto gratuito ou com desconto, servindo como prova de conceito e case de referência para expansão a outros restaurantes do mesmo perfil. A divulgação se apoia em canais especializados do setor, como associações de bares e restaurantes (ex.: Abrasel) e eventos do segmento, além de conteúdo digital demonstrando resultado orientado a dado como forma de gerar credibilidade junto a outros gestores.

Relacionamento com Clientes: o relacionamento é de acompanhamento próximo na fase inicial, com suporte dedicado durante a implantação e o piloto para calibrar sugestões do sistema à realidade do estabelecimento. Após a adoção, evolui para suporte contínuo e atualização periódica, reforçado por relatórios de resultado que mantêm o gestor engajado com o valor entregue ao longo do tempo.

Fontes de Receita: a receita principal vem de assinatura mensal (modelo SaaS), com valor definido pelo porte do restaurante — número de mesas e praças atendidas. O piloto inicial é oferecido sem custo ou com desconto, convertendo-se em assinatura plena após validação do resultado junto ao gestor. Receita complementar pode vir de suporte e customização adicionais para restaurantes com necessidade específica de configuração do sistema.

Recursos Principais: equipe de desenvolvimento com domínio de backend (ASP.NET Core), frontend (Angular) e ciência de dados (Python) para manter e evoluir os algoritmos de alocação, recomendação e BI. Base de dados operacional de cada restaurante (comandas, faturamento por praça e garçom), que alimenta os modelos analíticos e é o principal ativo de valor do sistema ao longo do tempo. Infraestrutura de hospedagem e processamento para rodar os modelos de programação linear e clusterização com desempenho adequado ao uso em tempo real no salão.

Atividades Principais: desenvolvimento e manutenção contínua do sistema (comandas, BI, algoritmos de alocação e recomendação), incluindo calibração periódica dos modelos com dado real de cada restaurante cliente — já que os pesos de alocação e os parâmetros de recomendação evoluem com o histórico acumulado. Suporte à implantação em cada novo restaurante, adaptando composição de praças e cardápio à realidade do estabelecimento. Garantia de conformidade contínua com a LGPD à medida que a base de dados cresce com múltiplos clientes.

Parcerias Principais: restaurantes parceiros para piloto de validação contínua dos modelos analíticos com dado real, essenciais para calibrar alocação e recomendação antes da expansão a novos clientes. Provedores de infraestrutura em nuvem para hospedagem e processamento dos algoritmos. Associações do setor, como a Abrasel, como canal de credibilidade e aproximação com potenciais clientes. Eventualmente, parceria com fornecedores de maquininhas/POS já usadas pelos restaurantes, para integração ou substituição gradual do sistema de lançamento de pedido hoje em uso.

Estrutura de Custos: os principais custos são de desenvolvimento e manutenção da equipe técnica (backend, frontend e ciência de dados) e de infraestrutura de hospedagem e processamento em nuvem, que crescem conforme a base de restaurantes clientes aumenta. Há também custo de suporte à implantação em cada novo restaurante, incluindo o período de piloto gratuito ou com desconto, e custo de conformidade contínua com a LGPD à medida que a base de dados se expande. O modelo é de custos majoritariamente fixos e recorrentes (equipe e infraestrutura), com pequena variação atrelada ao número de restaurantes atendidos.

3.2 VIABILIDADE FINANCEIRA E CUSTOS

Esta seção apresenta uma estimativa de custos para colocar o GASTRA em produção, com base nos preços públicos das ferramentas e serviços em nuvem vigentes em setembro de 2026. Os valores consideram um ambiente de demonstração/piloto — um restaurante cliente, tráfego moderado —, compatível com o modelo de negócio descrito na seção anterior antes de uma expansão a múltiplos clientes, quando os custos de computação escalariam proporcionalmente à base de restaurantes atendidos (conforme já indicado na Estrutura de Custos do Canvas).

A stack do GASTRA — backend em ASP.NET Core, serviço analítico em Python/FastAPI e banco de dados MySQL — mapeia diretamente para os serviços gerenciados da Microsoft Azure, provedor já usado como referência no ambiente de desenvolvimento do projeto (contêineres Docker equivalentes aos que rodam localmente). Cada um dos dois

serviços de backend roda em uma instância independente de Plano de Serviço de Aplicativo (App Service) na camada Básica (B1), voltada a cargas de trabalho de baixo tráfego, sem necessidade de dimensionamento automático, com o plano B1 custando aproximadamente US$ 13,14 por mês. O banco de dados MySQL utiliza o nível Flexible Server na camada Burstable (B1ms, 1 vCPU e 2 GiB de memória), com custo de aproximadamente US$ 12,41 por mês apenas pela instância de computação, mais um custo pequeno e variável de armazenamento. O frontend em Angular, por ser uma aplicação de página única, pode ser hospedado no nível gratuito do serviço de Aplicativos Web Estáticos do Azure, sem custo mensal para o volume de tráfego esperado num piloto. O domínio nacional (.com.br), registrado via Registro.br — entidade responsável pelo registro de domínios sob o .br no Brasil —, custa R$ 40,00 por ano, com renovação no mesmo valor. O certificado SSL para conexões seguras (https://) não gera custo adicional, pois o Azure App Service oferece suporte gratuito a certificados SNI SSL.

Considerando a cotação do dólar em torno de R$ 5,15 em setembro de 2026, a Tabela 1 resume a estimativa mensal:

Tabela 1 - Estimativa de custos mensais de infraestrutura

Item	**Serviço**	**Custo estimado (mensal)**
Backend (ASP.NET Core)	Azure App Service, plano B1	R$ 68,00
Serviço analítico (Python/ FastAPI)	Azure App Service, plano B1	R$ 68,00
Banco de Dados (MySQL)	Azure Database for MySQL Flexible Server, B1ms	R$ 64,00
Armazenamento do banco (≈20 GB)	Incluído no plano acima	R$ 12,00
Frontend (Angular)	Azure Static Web Apps, nível gratuito	R$ 0,00
Domínio (.com.br)	Registro.br	R$ 3,35
Certificado SSL	Azure App Services (SNI SSL)	R$ 0,00
Controle de versão e CI/CD	GitHub (repositório público, Actions)	R$ 0,00
Total estimado		**≈ R$ 215,00/mês**

Fonte: elaborado pelos autores, com base em Kuberns (2026), Pump.co (2025) e Registro.br (2026).

Ferramentas de desenvolvimento e modelagem usadas ao longo do projeto (Visual Studio Community, VS Code, brModelo, draw.io/Lucidchart) não entram nessa estimativa por serem gratuitas ou já licenciadas no ambiente acadêmico, não representando custo recorrente de operação do sistema em produção.

Limitações da estimativa: os valores de nuvem são calculados pela Microsoft em dólar e convertidos mensalmente pela cotação vigente, sujeitos a variação cambial; a tabela reflete apenas o custo de infraestrutura em regime de baixo tráfego (piloto/demonstração) e não contempla custos adicionais que surgiriam com escala — múltiplos restaurantes clientes, maior volume de requisições ao serviço analítico, ou necessidade de camadas com dimensionamento automático (Standard/Premium), que elevam consideravelmente o custo por instância.

4 ENGENHARIA DE SOFTWARE E PROJETO DO SISTEMA

Este capítulo apresenta os artefatos de engenharia de software produzidos para o GASTRA, na ordem em que um depende do outro: os requisitos levantados na pesquisa de campo (seção 4.1), os casos de uso e as histórias de usuário que os detalham (seção 4.2), a modelagem de dados, do modelo conceitual ao banco implementado (seção 4.3), e a arquitetura da solução, com o protótipo que antecedeu as telas definitivas (seção 4.4). Todos os artefatos citados estão versionados no repositório do projeto, junto do código que os implementa, e foram revisados contra esse código antes de entrarem neste documento: quando o sistema e a documentação divergiram, a divergência foi registrada e corrigida em um dos dois lados, e não apenas neste texto.

4.1 LEVANTAMENTO E ESPECIFICAÇÃO DE REQUISITOS

Os requisitos do GASTRA resultam do cruzamento das duas frentes de levantamento descritas na seção 1.5 — os questionários aplicados a clientes (n = 13) e a garçons (n = 2) e a entrevista semiestruturada com um garçom de restaurante à la carte — com o escopo definido no projeto de pesquisa. Seguindo a distinção proposta por Sommerville (2019), eles foram separados em requisitos funcionais, que descrevem o que o sistema deve fazer, requisitos não funcionais, que restringem como ele deve fazê-lo, e regras de negócio, que registram as políticas do domínio que o sistema precisa respeitar independentemente da tela ou da funcionalidade em que apareçam.

Alguns achados da pesquisa de campo orientaram diretamente a especificação. Entre os clientes, 77% (10 de 13) relataram dificuldade para identificar opções compatíveis com restrições alimentares no cardápio, o que deu origem ao cardápio digital com sinalização dietética (RF05) e ao registro da restrição informada pelo cliente (RF14); 85% (11 de 13) declararam-se confortáveis com o uso de histórico de consumo desde que anonimizado, condição que fundamenta a recomendação de pratos (RF09) sob a restrição de não usar atributo pessoal (RN05). Entre os garçons, a amostra de dois respondentes não sustenta porcentagem e foi tratada como indício qualitativo, aprofundado na entrevista: os dois perceberam diferença de potencial de faturamento entre as praças, e a entrevista confirmou que a distribuição atual depende de hierarquia informal — evidência que sustenta a alocação por programação linear (RF06, RN03) e o índice de desempenho que não se limita ao volume de venda (RF11).

Cada requisito foi registrado com identificador, descrição, fonte, artefatos relacionados, método de verificação e situação na Matriz de Rastreabilidade, que liga o

requisito aos casos de uso, às histórias de usuário e às entidades do modelo de dados. O processo iterativo adotado (seção 1.5) fez a especificação evoluir ao longo do desenvolvimento: os requisitos RF23 a RF25 e as regras RN06 a RN09 não vieram do levantamento inicial, mas de decisões da dupla tomadas durante as sprints, e a coluna de fonte registra essa origem de forma explícita, para distinguir o que foi pedido pelos usuários do que foi decidido no projeto.

4.1.1 Requisitos funcionais

Foram especificados 25 requisitos funcionais, dos quais 24 foram aprovados e um foi reprovado, conforme o Quadro 1.

Quadro 1 – Requisitos funcionais do GASTRA

ID	Descrição	Ator	Fonte
RF01	Abrir uma comanda vinculada a uma mesa	Garçom	Núcleo do módulo de comandas
RF02	Sugerir a composição da mesa, que o garçom confirma ou ajusta	Garçom	Decisão de projeto e entrevista
RF03	Registrar os itens do pedido na comanda aberta, com lista de pendências visível	Garçom	Núcleo de comandas e entrevista
RF04	Fechar a comanda, calculando o total com ou sem taxa de serviço, a pedido do cliente	Garçom	Núcleo do módulo de comandas
RF05	Consultar o cardápio digital por QR code, com fotos, preço e sinalização dietética, sem função de pedido	Cliente	Questionário de clientes e entrevista
RF06	Calcular uma sugestão de alocação de garçons às praças por programação linear (RN03)	Sistema, Metre	Escopo original
RF07	Visualizar e ajustar a alocação sugerida antes de confirmá-la	Metre	Escopo original e entrevista
RF08	Registrar o histórico de alocações e de faturamento por praça, para a RN03 e para o BI	Sistema	Questionário de garçons e entrevista
RF09	Sugerir combinações de pratos ao garçom a partir de padrões de consumo	Garçom	Escopo original
RF10	Exibir relatórios agregados de faturamento por praça e por garçom, com os KPIs definidos	Gerente	Escopo original
RF11	Exibir um índice composto de desempenho dos garçons, que não se limite ao volume de venda	Gerente, Garçom	Questionário de garçons e entrevista
RF12	Permitir ao cliente solicitar a exclusão do seu histórico de pedidos (**reprovado**)	Cliente	Questionário de clientes (LGPD)
RF13	Consultar a própria comanda em tempo real: itens lançados e valor parcial	Cliente	Entrevista
RF14	Registrar a restrição alimentar informada voluntariamente pelo cliente, com categoria e observação livre ligadas à comanda ativa	Garçom	Decisão da dupla, informada pela entrevista

ID	Descrição	Ator	Fonte
RF15	Autenticar-se com e-mail e senha	Usuário	Decisão da dupla
RF16	Exigir segunda etapa de verificação por aplicativo autenticador (TOTP) para Gerente e Coordenador	Gerente, Coordenador	Decisão da dupla
RF17	Encerrar a sessão	Usuário	Decisão da dupla
RF18	Cadastrar, editar, inativar e reativar contas, redefinir senha e reiniciar a verificação em duas etapas de outra conta	Gerente	Decisão da dupla
RF19	Cadastrar itens do cardápio com nome, categoria, preço, descrição e sinalização dietética	Gerente, Coordenador	Decisão da dupla
RF20	Atualizar o preço de um item do cardápio	Gerente, Coordenador	Decisão da dupla
RF21	Marcar um item do cardápio como disponível ou indisponível	Gerente, Coordenador	Decisão da dupla
RF22	Criar e remover promoções vinculadas a um ou mais itens do cardápio	Gerente, Coordenador	Decisão da dupla
RF23	Cadastrar e editar praças e mesas, vinculando cada mesa a uma praça	Gerente	Decisão da dupla (16/09)
RF24	Marcar um item do pedido como entregue ou cancelá-lo com justificativa de uma lista fechada	Garçom	Decisão da dupla (16/09), necessária à RN02
RF25	Avaliar o atendimento depois do fechamento da conta, com nota de 1 a 5 e comentário opcional	Cliente	Decisão da dupla (21/09)

Fonte: elaborado pelos autores.

O RF12 merece registro à parte, porque a sua reprovação é também um resultado da modelagem. Ele nasceu do questionário de clientes, na parte dedicada à Lei Geral de Proteção de Dados, como o direito de o cliente pedir a exclusão do seu histórico de pedidos. Ao longo da modelagem, porém, decidiu-se que o sistema não identifica o cliente em momento algum: a comanda identifica a mesa, e não a pessoa, e não há cadastro, nome, documento ou contato de cliente em nenhuma tabela (RN04, RN08). Sem identificação, não existe um histórico de cliente a ser excluído, e o requisito perde o objeto. A preocupação que o originou — o controle do titular sobre os próprios dados — foi atendida de forma mais forte pelo princípio da necessidade previsto na LGPD (BRASIL, 2018): o dado pessoal do cliente não é coletado, em vez de ser coletado e depois excluído a pedido. A linha do RF12 foi mantida na Matriz de Rastreabilidade com a situação "reprovado" e o motivo, e os requisitos seguintes não foram renumerados, para não quebrar as referências cruzadas entre os documentos.

4.1.2 Requisitos não funcionais

Os cinco requisitos não funcionais, apresentados no Quadro 2, cobrem desempenho, usabilidade, privacidade, segurança e disponibilidade.

Quadro 2 – Requisitos não funcionais do GASTRA

ID	Descrição	Categoria	Fonte
RNF01	Refletir um novo pedido registrado na comanda em, no máximo, 2 segundos	Desempenho	Questionário de clientes, entrevista e Nielsen (1993)
RNF02	Abrir mesa e registrar um item em, no máximo, 5 toques: 2 para abrir a mesa e 3 para lançar o item	Usabilidade	Questionário de clientes e Nielsen (1993)
RNF03	Armazenar dados pessoais do cliente com minimização e acesso restrito por papel	Privacidade (LGPD)	Decisão de projeto
RNF04	Dar a cada papel (Gerente, Coordenador, Metre, Garçom) permissões de leitura e escrita distintas	Segurança	Decisão de projeto
RNF05	Manter o núcleo de comandas operacional durante todo o horário de funcionamento	Disponibilidade	Decisão de escopo

Fonte: elaborado pelos autores.

Os limites do RNF01 e do RNF02 foram definidos a partir dos limites de tempo de resposta descritos por Nielsen (1993): até cerca de um segundo, o fluxo de pensamento do usuário não é interrompido, e até cerca de dez segundos a atenção ainda se mantém na tarefa. No salão, o garçom opera o sistema entre uma mesa e outra, muitas vezes com o cliente à frente, e por isso os dois requisitos buscam manter o registro do pedido dentro da faixa em que ele não percebe espera nem precisa pensar na interface. O RNF05 teve consequência direta na arquitetura, como se verá na seção 4.4: o núcleo de comandas não depende do serviço analítico para funcionar.

4.1.3 Regras de negócio

As nove regras de negócio estão no Quadro 3. As regras RN01 a RN05 vieram do levantamento e da modelagem do domínio; as regras RN06 a RN09 registram decisões de segurança e privacidade tomadas durante o desenvolvimento.

Quadro 3 – Regras de negócio do GASTRA

ID	Regra	Fonte
RN01	Composição da mesa: Solo (1), Casal (2), Grupo pequeno (3 a 4, sem item infantil), Família (3 ou mais, com item infantil) e Grupo grande (5 ou mais)	Decisão de projeto e entrevista
RN02	A comanda só pode ser fechada se todos os itens estiverem entregues ou cancelados com justificativa; itens pendentes ficam visíveis como lembrete	Dupla e entrevista
RN03	A alocação equilibra o faturamento médio por turno de cada garçom nos últimos 30 dias, ponderado pelo potencial da praça, e considera o tempo desde a última praça de alto potencial,	Proposta da dupla, com evidência do questionário e da entrevista

ID	Regra	Fonte
	em soma ponderada com w1 = 0,6 e w2 = 0,4, pesos calibrados por simulação	
RN04	Dados do cliente só são acessíveis ao Garçom e ao Metre, no contexto da mesa ativa; fora disso, apenas de forma agregada ou anonimizada	Decisão de projeto (LGPD)
RN05	O sistema não captura, infere nem armazena atributos sensíveis do cliente para segmentação	Decisão de projeto (LGPD)
RN06	Senhas são armazenadas apenas em hash, e o login compara somente hashes	Decisão da dupla
RN07	O código do segundo fator segue a RFC 6238 (janela de 30 segundos); a chave secreta é gerada uma única vez e nunca é reexibida	Decisão da dupla
RN08	A avaliação do atendimento é anônima, uma por comanda, só depois do fechamento e em até 24 horas; o Gerente vê apenas números agregados, e a média por garçom só entra no índice de desempenho com pelo menos 5 avaliações no período	Decisão da dupla (21/09)
RN09	Cinco tentativas de acesso erradas seguidas, de senha ou de código, bloqueiam a conta por 15 minutos; durante o bloqueio, o login responde a mesma mensagem da senha errada	Decisão da dupla (23/09)

Fonte: elaborado pelos autores.

4.2 MODELAGEM DO SISTEMA (CASOS DE USO / HISTÓRIAS DE USUÁRIO)

Os requisitos funcionais foram detalhados em 25 casos de uso, organizados em seis grupos, cada um com o seu diagrama na notação da UML (Quadro 4). Os atores são os quatro papéis de funcionário — Gerente, Coordenador, Metre e Garçom, especializações de um ator genérico Usuário, que concentra os casos de uso de acesso — e o Cliente, que usa o sistema sem autenticação, pelo código de acesso da sua mesa.

Quadro 4 – Casos de uso do GASTRA por grupo

Grupo	Casos de uso	Atores
Autenticação e controle de acesso	UC01 Autenticar-se; UC02 Confirmar segundo fator; UC03 Encerrar sessão; UC04 Gerenciar contas de usuário	Usuário, Gerente, Coordenador
Gestão de cardápio	UC05 Cadastrar item; UC06 Atualizar preço; UC07 Marcar item disponível ou indisponível; UC08 Criar promoção; UC09 Remover promoção	Gerente, Coordenador
Núcleo de comandas	UC10 Abrir comanda; UC11 Confirmar ou ajustar composição da mesa; UC12 Registrar item de pedido; UC13 Registrar restrição alimentar; UC14 Fechar comanda; UC23 Atualizar situação do item	Garçom
Blocos analíticos	UC15 Gerar sugestão de alocação; UC16 Visualizar relatórios de BI; UC17 Visualizar índice de desempenho; UC18 Visualizar sugestão de pratos; UC21 Confirmar alocação; UC22 Ajustar alocação	Metre, Gerente, Garçom

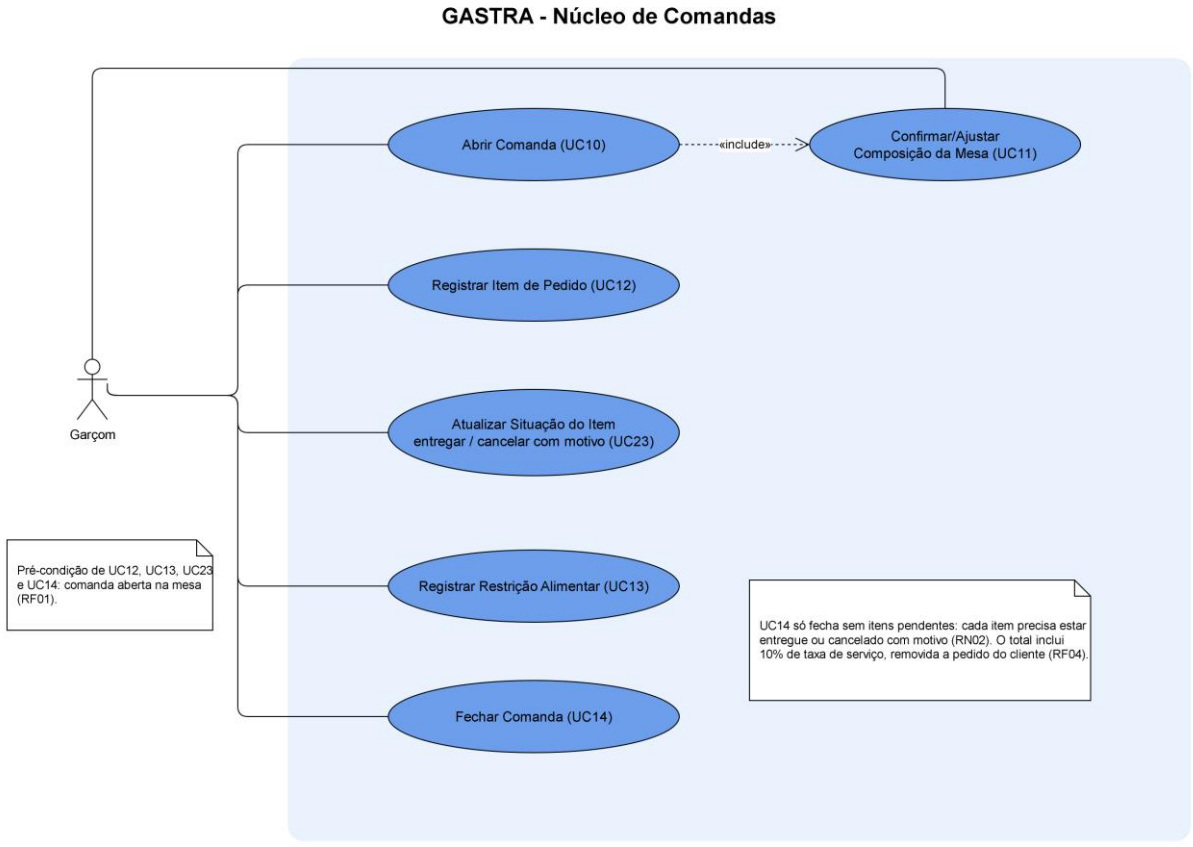
Grupo	Casos de uso	Atores
Cliente	UC19 Consultar cardápio digital; UC20 Consultar comanda em tempo real; UC25 Avaliar o atendimento	Cliente
Configuração do salão	UC24 Gerenciar praças e mesas	Gerente

Fonte: elaborado pelos autores.

A numeração dos casos de uso não segue a ordem dos grupos porque acompanha a ordem em que foram especificados: o UC23 surgiu com o RF24, quando se percebeu que a regra de fechamento (RN02) exigia marcar a entrega e o cancelamento de cada item, e o UC25 surgiu com a avaliação do atendimento (RF25). Pelo mesmo motivo do RF12, os números não foram reorganizados depois.

A Figura 2 apresenta o diagrama do núcleo de comandas, que concentra a operação diária do salão.

Figura 2 – Diagrama de casos de uso do núcleo de comandas



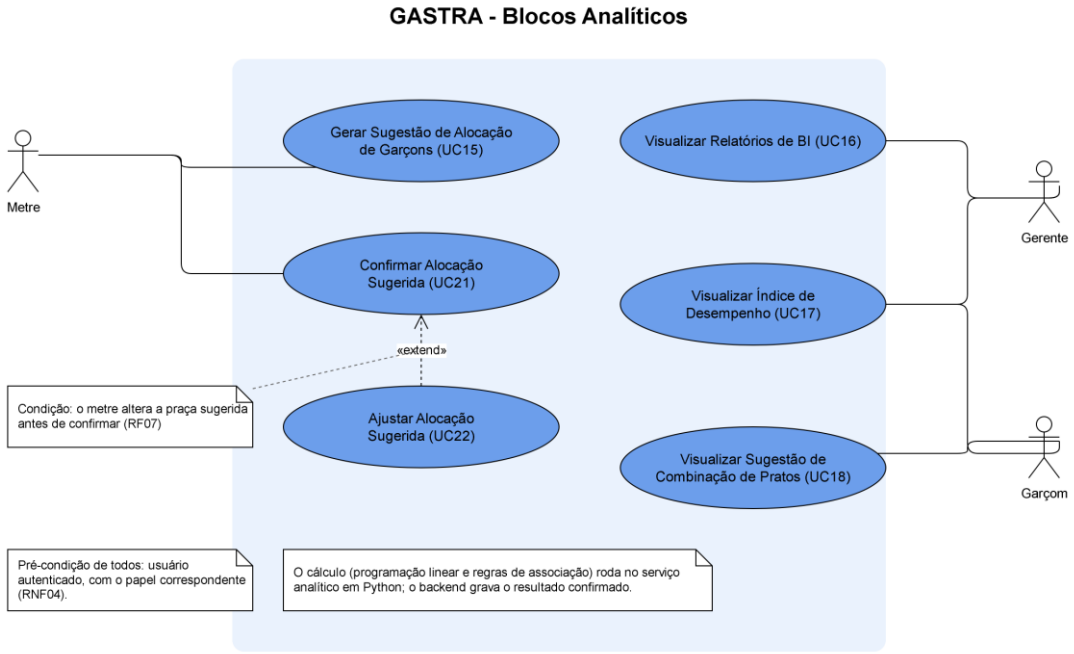
Fonte: elaborado pelos autores.

Dois relacionamentos do diagrama merecem destaque. A confirmação da composição da mesa (UC11) é incluída na abertura da comanda (UC10), porque toda abertura termina com a composição sugerida pela RN01 confirmada ou ajustada pelo garçom; ela também aparece

associada diretamente ao Garçom, porque a composição pode ser corrigida depois, com a comanda já aberta — por exemplo, quando chega mais uma pessoa à mesa. Já o lançamento de um item infantil pode reclassificar a composição automaticamente para Família, desde que o garçom não a tenha ajustado à mão: o ajuste manual prevalece sobre a regra automática.

Os blocos analíticos, na Figura 3, reúnem os casos de uso que dependem de dado histórico: a alocação de garçons, com geração, ajuste e confirmação separados, porque o Metre mantém a decisão final sobre a sugestão do sistema (RF07), e os relatórios, o índice de desempenho e a sugestão de pratos.

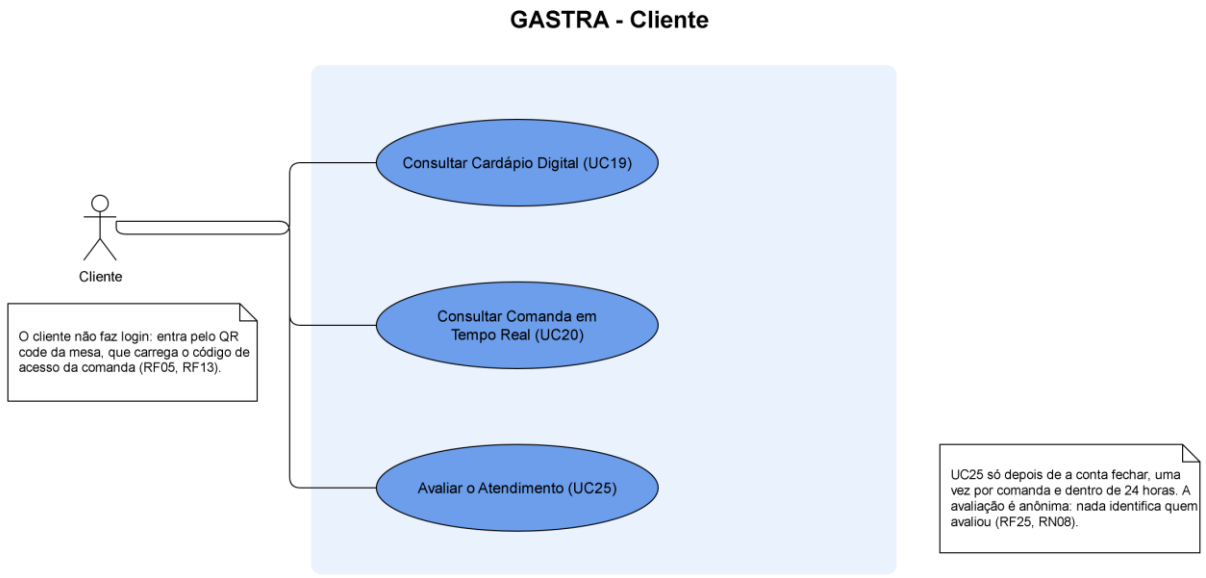
Figura 3 – Diagrama de casos de uso dos blocos analíticos



Fonte: elaborado pelos autores.

O Cliente, na Figura 4, é o único ator que não se autentica. Ele acessa o cardápio digital (UC19) e a conta da própria mesa (UC20) pelo QR code, e, depois do fechamento, pode avaliar o atendimento (UC25). O código de acesso funciona como a senha da comanda: dá acesso somente àquela conta e nunca é gravado em log.

Figura 4 – Diagrama de casos de uso do Cliente



Fonte: elaborado pelos autores.

Os diagramas dos demais grupos — autenticação, gestão de cardápio e configuração do salão — seguem a mesma notação e estão no repositório do projeto.

Os requisitos também foram escritos como 24 histórias de usuário, no formato "Como [papel], quero [ação], para [benefício]", que registra, além do que o sistema deve fazer, quem se beneficia e por quê. Cada história está ligada ao requisito funcional de origem e recebeu critérios de aceite, apresentados no Quadro 5. Para cada critério, indica-se o teste automatizado que o comprova, pelo nome da classe e do método no código, de modo que a verificação possa ser repetida a qualquer momento: se a regra deixar de valer, o teste deixa de passar. Os testes e os seus resultados são discutidos no capítulo 5.

Quadro 5 – Histórias de usuário, critérios de aceite e testes que os comprovam

História	**Critérios de aceite**	**Teste que comprova**
US01 — Como Metre, quero receber uma sugestão automática de alocação (RF06)	Cada garçom presente recebe exatamente uma praça, sem exceder as vagas; a praça de maior potencial vai para quem faturou menos por turno	`AlocacaoControllerTests.Sugestao_EnviaAoPythonOsFatoresDaRN03_EGravaOResultado`; `test_alocacao::test_praca_de_alto_potencial_vai_para_quem_faturou_menos`
US02 — Como Metre, quero ajustar a alocação antes de confirmá-la (RF07)	Antes da confirmação, é possível mudar um garçom de praça; em praça cheia, a mudança exige uma troca; depois da confirmação, nenhum ajuste é aceito	`AlocacaoControllerTests.Ajuste_TrocaAPraca_ERegistraSugeridaEEscolhida`; `AlocacaoControllerTests.Ajuste_DepoisDeConfirmado_Retorna422`

História	Critérios de aceite	Teste que comprova
US03 — Como Gerente, quero o histórico de faturamento por praça e por garçom (RF08)	O potencial da praça é o faturamento médio por turno, incluindo praças sem movimento; só alocações confirmadas contam como histórico	`ObjetosDoBancoTests.Faturamento_medio_da_praca_e_calculado_por_turno_e_inclui_praca_sem_movimento`; `AlocacaoControllerTests.Confirmacao_TravaOTurno_ERegistraNaAuditoria`
US04 — Como Garçom, quero sugestões de combinação de pratos (RF09)	Só são sugeridos itens disponíveis que ainda não estão na comanda; se o serviço analítico estiver fora do ar, a comanda continua funcionando sem sugestão	`SugestoesComandaTests.EnviaAoServicoSoOQuePodeSerOferecido`; `SugestoesComandaTests.ServicoFora_Responde200ComListaVazia_ESemTravarAComanda`
US05 — Como Gerente, quero relatórios agregados com os KPIs (RF10)	Sem filtro, o período são os últimos 30 dias; todas as praças aparecem, inclusive as sem movimento; os relatórios são exclusivos do Gerente e a consulta é auditada	`IndicadoresControllerTests.RelatorioDePracas_MostraTodasAsPracas_InclusiveSemMovimento`; `IndicadoresControllerTests.Relatorios_SaoDoGerente_ERegistramAConsulta`
US06 — Como Garçom, quero um índice de desempenho que não seja só venda (RF11)	O índice é calculado por turno trabalhado; o garçom vê apenas a própria posição; a avaliação do cliente só entra com o mínimo de avaliações	`IndiceDeDesempenhoTests.ComparaPorTurno_QuemTrabalhouMaisNaoGanhaSoPorIsso`; `IndicadoresControllerTests.Ranking_OGarcomVeSoAPropriaPosicao_SemNomeDosColegas`
US07 — Como Garçom, quero abrir uma comanda vinculada a uma mesa (RF01)	Com mesa e quantidade de pessoas, a comanda nasce com a composição sugerida e o código de acesso do cliente; mesa inexistente é recusada	`ComandaControllerTests.Abrir_ComMesaEPessoas_Retorna201ComComposicaoSugeridaECodigoDeAcesso`; `ComandaControllerTests.Abrir_ComMesaInexistente_Retorna404`
US08 — Como Garçom, quero que o sistema sugira a composição da mesa (RF02)	A composição segue a RN01 pela quantidade de pessoas; item infantil em mesa de três ou mais muda para Família; o ajuste manual impede a reclassificação	`ComandaTests.Abrir_SugereAComposicaoPelaQuantidadeDePessoas`; `ComandaTests.AjustarComposicaoNaMao_CongelaARegraAutomatica`
US09 — Como Garçom, quero registrar itens com lista de pendências (RF03)	O item guarda o preço do momento, que não muda se o cardápio mudar; item indisponível não pode ser lançado; os pendentes aparecem destacados	`ComandaTests.AdicionarItem_CopiaOPrecoDoMomentoENaoMudaDepois`; `ComandaControllerTests.LancarItem_Indisponivel_Retorna422`
US10 — Como Garçom, quero fechar a comanda com o total calculado (RF04)	O total soma 10% de taxa de serviço, removível a pedido do cliente; com item pendente, o fechamento é recusado (RN02)	`ComandaControllerTests.Fechar_ComItemEntregue_CalculaTotalComTaxaDeDezPorCento`; `ComandaControllerTests.`

História	Critérios de aceite	Teste que comprova
		`Fechar_ComItemPendente_Retorna422`
US11 — Como Garçom, quero registrar a restrição alimentar informada pelo cliente (RF14)	A restrição guarda categoria e observação livre; no fechamento, a observação é apagada e só a categoria fica; o Gerente nunca a vê	`ComandaControllerTests.Fechar_ApagaAObservacaoLivreDaRestricaoEMantemACategoria`; `ComandaControllerTests.Restricao_GerenteNaoVe_NemNaConsultaNemNoPainel`
US12 — Como Cliente, quero consultar o cardápio digital (RF05)	O cardápio abre sem login; item indisponível sai do cardápio digital, mas continua na gestão	`CardapioControllerTests.CardapioDigital_SemLogin_Retorna200`; `CardapioControllerTests.MarcarIndisponivel_ItemSaiDoCardapioDigitalMasContinuaNaGestao`
US13 — Como Cliente, quero consultar minha comanda em tempo real (RF13)	A conta abre pelo código de acesso, sem login, e não mostra a restrição alimentar; código inexistente não revela nada	`ComandaControllerTests.ConsultaDoCliente_PeloCodigoDeAcesso_FuncionaSemLoginENaoExpoeRestricao`; `ComandaControllerTests.ConsultaDoCliente_ComCodigoInexistente_Retorna404`
US14 — Como Usuário, quero me autenticar com e-mail e senha (RF15)	Senha errada e e-mail inexistente recebem a mesma mensagem; cinco erros seguidos bloqueiam a conta, inclusive para a senha certa (RN09)	`AutenticacaoControllerTests.Login_ComSenhaErradaOuEmailInexistente_Retorna401ComAMesmaMensagem`; `AutenticacaoControllerTests.Login_CincoSenhasErradas_BloqueiaAtéASenhaCertaComAMesmaMensagem`
US15 — Como Gerente ou Coordenador, quero confirmar o login com o autenticador (RF16)	O Gerente não recebe acesso só com a senha; a confirmação do código libera o acesso; a chave não pode ser gerada de novo	`AutenticacaoControllerTests.Login_Gerente_NaoRecebeTokenDeAcessoEPrecisaConfigurarOAutenticador`; `AutenticacaoControllerTests.SegundoFator_FluxoCompleto_LiberaAcessoEOSegredoNaoPodeSerGeradoDeNovo`
US16 — Como Usuário, quero encerrar minha sessão (RF17)	Depois do logoff, o token deixa de valer no servidor, e não apenas no navegador	`AutenticacaoControllerTests.Logoff_InvalidaOTokenNoServidor`
US17 — Como Gerente, quero gerenciar as contas de usuário (RF18)	A conta cadastrada consegue entrar; a troca de papel derruba o token antigo; a inativação bloqueia o acesso na hora, e a reativação o devolve	`UsuarioControllerTests.Cadastrar_ComDadosValidos_Retorna201EOUsuarioConsegueEntrar`; `UsuarioControllerTests.Inativar_`

História	Critérios de aceite	Teste que comprova
		`BloqueiaTokenELogin_EReativarDevolveOAcesso`
US18 — Como Gerente ou Coordenador, quero cadastrar itens do cardápio (RF19)	Dados válidos criam o item; nome vazio e preço zero são recusados com as duas mensagens; o Garçom não cadastra	`CardapioControllerTests.Cadastrar_ComDadosValidos_Retorna201ComOItem`; `CardapioControllerTests.Cadastrar_ComNomeVazioEPrecoZero_Retorna400ComOsDoisErrosEmPortugues`
US19 — Como Gerente ou Coordenador, quero atualizar o preço de um item (RF20)	O preço muda no cardápio; comandas já lançadas mantêm o preço do momento	`CardapioControllerTests.AtualizarPreco_DeItemExistente_Retorna204EOPrecoMuda`; `ComandaTests.AdicionarItem_CopiaOPrecoDoMomentoENaoMudaDepois`
US20 — Como Gerente ou Coordenador, quero marcar um item como indisponível (RF21)	O item sai do cardápio digital e não pode ser lançado, mas continua na gestão	`CardapioControllerTests.MarcarIndisponivel_ItemSaiDoCardapioDigitalMasContinuaNaGestao`; `ComandaControllerTests.LancarItem_Indisponivel_Retorna422`
US21 — Como Gerente ou Coordenador, quero criar e remover promoções (RF22)	O preço promocional aparece no cardápio e é o lançado na comanda; o desconto nunca deixa o item de graça; remover desativa sem apagar	`PromocaoControllerTests.ItemLancadoNaComanda_SaiComOPrecoPromocional`; `PromocaoTests.AplicarDesconto_NuncaDeixaOItemDeGraca`; `PromocaoControllerTests.Remover_DesativaSemApagar_EOPrecoVoltaAoNormal`
US22 — Como Gerente, quero cadastrar praças e mesas (RF23)	Toda mesa pertence a uma praça; mesa cadastrada pelo Gerente já pode receber comanda	`SalaoControllerTests.CadastrarMesa_VinculaAPracaEAparecenaListagem`; `SalaoControllerTests.MesaCadastradaPeloGerente_PermiteAoGarcomAbrirComanda`
US23 — Como Garçom, quero marcar cada item como entregue ou cancelá-lo com motivo (RF24)	O cancelamento exige um motivo da lista fechada; o item cancelado sai da conta e não impede o fechamento	`ComandaControllerTests.CancelarItem_SemMotivo_Retorna400`; `ComandaControllerTests.CancelarItem_ComMotivo_TiraOItemDaConta`
US24 — Como Cliente, quero avaliar o atendimento depois de pagar (RF25)	A avaliação só é aceita com a conta fechada e uma única vez; nada identifica quem avaliou	`ComandaControllerTests.Avaliacao_DepoisDeFechar_AceitaSemLoginENaoDeixaAvaliarDeNovo`; `ComandaControllerTests.Avaliacao_ComContaAberta_Recusa`;

História	Critérios de aceite	Teste que comprova
		`ComandaControllerTests.Avaliacao_NaoIdentificaQuemAvaliou`

Fonte: elaborado pelos autores.

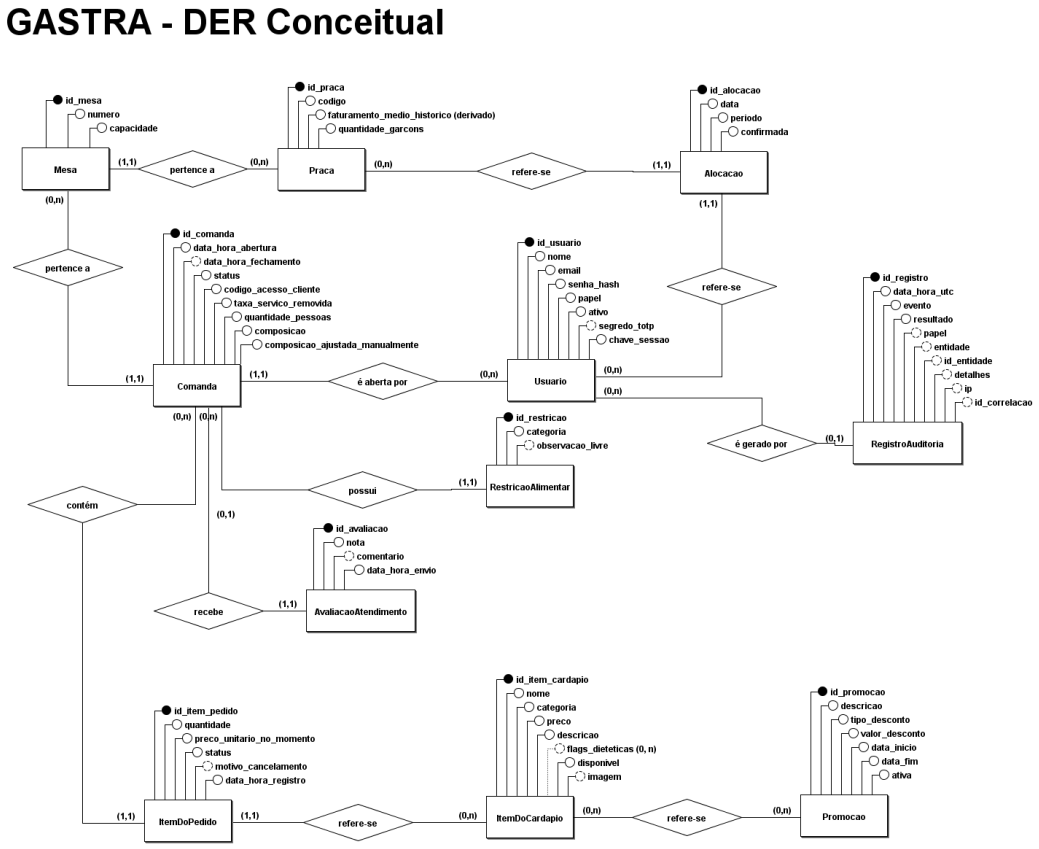
.3 MODELAGEM DE DADOS (MER / DER)

A modelagem de dados percorreu os três níveis clássicos de abstração: o modelo conceitual, que descreve o domínio independentemente de tecnologia; o modelo lógico, que o traduz em tabelas, chaves e relacionamentos; e o modelo físico, que é o banco efetivamente criado no MySQL. No GASTRA, o modelo físico não foi escrito à mão: ele é gerado a partir das classes do domínio pelas migrations do Entity Framework Core (abordagem Code First, seção 2.2). Por isso, o caminho de volta — do banco implementado ao modelo conceitual — foi percorrido por engenharia reversa, para garantir que o banco criado pelo código corresponde ao que foi modelado.

4.3.1 Modelo conceitual

O Modelo de Entidade-Relacionamento (MER) do GASTRA foi descrito em linguagem natural, com 11 entidades e 11 relacionamentos, cada relacionamento com cardinalidade, participação, entidades envolvidas e a regra de negócio que o justifica. A partir dele foi desenhado o Diagrama Entidade-Relacionamento (DER) na ferramenta brModelo, apresentado na Figura 5.

Figura 5 – Diagrama Entidade-Relacionamento conceitual do GASTRA



Fonte: elaborado pelos autores no brModelo.

Algumas escolhas de modelagem refletem diretamente as regras de negócio. O item do pedido e a alocação são entidades associativas: o primeiro liga a comanda ao item do cardápio e guarda o preço no momento do pedido, que não pode acompanhar mudanças posteriores do cardápio; a segunda liga o garçom à praça em um turno e guarda se a alocação foi confirmada, porque só as alocações confirmadas entram no histórico usado pela RN03. O faturamento médio histórico da praça é um atributo derivado: ele é calculado a partir das comandas fechadas e nunca armazenado, para não divergir da soma que o origina. As sinalizações dietéticas do item do cardápio são um atributo multivalorado, e a avaliação do atendimento é uma entidade com identificador próprio que depende da existência da comanda — não há avaliação sem comanda —, dependência expressa pela participação obrigatória, com cardinalidade (1,1) do lado da avaliação e (0,1) do lado da comanda.

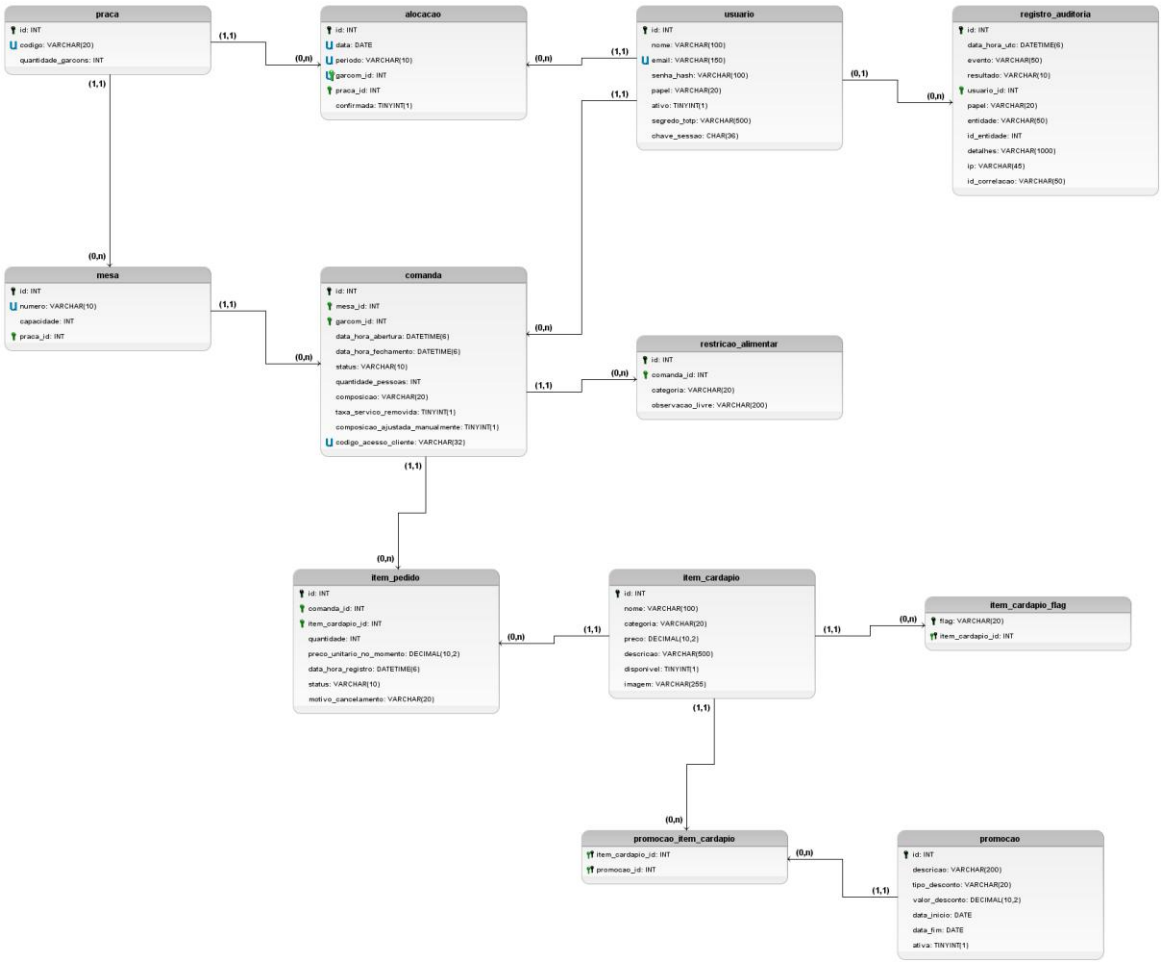
Destaca-se também o que o modelo não tem: não existe entidade Cliente. A comanda identifica a mesa, e não a pessoa, e essa ausência é a principal decisão de privacidade do projeto, discutida na seção 4.3.4.

4.3.2 Modelos lógico e físico

O modelo físico resulta das migrations do Entity Framework Core, e cada alteração do banco corresponde a uma migration versionada e revisada junto do código que a motivou. No estado atual, o banco tem 13 tabelas: uma para cada uma das 11 entidades do modelo conceitual, uma para o atributo multivalorado das sinalizações dietéticas e uma para o relacionamento N:N entre promoções e itens do cardápio. As tabelas estão ligadas por 13 chaves estrangeiras.

Para validar a correspondência entre o banco e o modelo conceitual, o catálogo do MySQL foi extraído por consulta às tabelas de metadados, comparado automaticamente com o DER exportado do brModelo e remontado como modelo lógico no próprio brModelo, apresentado na Figura 6. As diferenças encontradas foram classificadas em justificadas — como chaves estrangeiras, que só existem no nível físico, e o atributo derivado, que não vira coluna — e em divergências reais. Estas foram corrigidas: onze atributos que o banco aceita vazios passaram a aparecer como opcionais no DER, e a tabela da avaliação do atendimento, criada depois da primeira versão do modelo, foi incorporada ao MER e ao DER.

Figura 6 – Modelo lógico obtido por engenharia reversa do banco implementado



Fonte: elaborado pelos autores no brModelo, a partir do catálogo do MySQL.

Além das tabelas, o banco tem seis views e dois triggers. As views entregam consultas agregadas — faturamento por comanda, por praça e turno, faturamento médio por praça, desempenho do garçom por turno, faturamento por item e itens por comanda — usadas pelos relatórios de Business Intelligence e pelo serviço analítico, sem expor nenhum dado pessoal, o que é verificado por um teste automatizado. Os dois triggers protegem a tabela de auditoria: um impede qualquer alteração de registro, e o outro impede a exclusão de registros com menos de seis meses, prazo de retenção definido na política de log. Seguindo a decisão descrita na seção 2.2, não foram usadas stored procedures: as regras de negócio ficam no domínio da aplicação, onde são testadas sem depender de um banco ativo.

4.3.3 Normalização

Cada tabela foi verificada quanto à primeira, à segunda e à terceira formas normais, a partir das dependências funcionais derivadas das regras de negócio. Todos os valores são

atômicos, e o atributo multivalorado das sinalizações dietéticas ocupa tabela própria (1FN). Todas as tabelas, exceto duas tabelas associativas sem atributos fora da chave, têm chave primária de uma única coluna, o que elimina a possibilidade de dependência parcial (2FN). Nenhum total é armazenado: subtotal, taxa de serviço e total da comanda, assim como o faturamento médio da praça, são calculados no domínio ou nas views, o que evita dependências transitivas e o risco de o valor gravado divergir da soma dos itens (3FN).

Houve uma exceção consciente. Com a situação da comanda reduzida a "aberta" e "fechada", a coluna de situação passou a ser deduzível da data de fechamento — vazia significa aberta —, o que, em sentido estrito, fere a terceira forma normal. A coluna foi mantida por legibilidade das consultas de BI e por desempenho, já que o índice por mesa e situação atende à consulta mais frequente do salão, a das comandas abertas. O risco próprio de uma redundância, que é as duas colunas se contradizerem, foi eliminado por uma restrição CHECK no banco, que exige que as duas concordem, e pelo domínio, em que ambas só mudam juntas no fechamento. Outros casos que aparentam redundância não o são: o preço guardado no item do pedido é o preço no momento do pedido, e não uma cópia do preço atual; e o papel gravado na auditoria é o papel no momento da ação, que precisa sobreviver a uma mudança de papel do usuário.

4.3.4 Proteção de dados na modelagem

A modelagem partiu do princípio da necessidade previsto na LGPD (BRASIL, 2018): o cliente é anônimo para o sistema. Não há nome, documento, telefone ou e-mail de cliente em nenhuma tabela, e os dados pessoais que existem são, em sua maioria, dos funcionários. Cada coluna que guarda dado pessoal, ou que poderia tornar alguém identificável, foi classificada e recebeu uma proteção correspondente, conforme o Quadro 6.

Quadro 6 – Mapa de dados pessoais e proteção correspondente

Dado	**Classificação**	**Titular**	**Proteção**
Nome e e-mail do usuário	Pessoal	Funcionário	Só o Gerente gerencia contas; nunca vão para view nem para log
Hash da senha, segredo do autenticador e chave de sessão	Credencial	Funcionário	Hash BCrypt; segredo criptografado e nunca reexibido; chave de sessão trocada no logoff, na mudança de papel e na inativação
IP no registro de auditoria	Pessoal	Funcionário	Só em eventos de autenticação; retenção de seis meses garantida pelo trigger
Observação livre da restrição alimentar	Sensível (pode descrever condição de saúde)	Cliente	Opcional; nunca vai para log nem para view; apagada no fechamento da comanda

Dado	Classificação	Titular	Proteção
Categoria da restrição alimentar	Sensível na mesa ativa, sem titular identificável depois	Cliente	Visível só a Garçom e Metre com a comanda aberta (RN04); depois do fechamento, fica apenas a categoria, ligada a uma comanda anônima
Código de acesso da comanda	Credencial	Cliente	Dá acesso somente àquela conta; nunca vai para log
Comentário da avaliação	Pode tornar-se pessoal, se o cliente escrever dado próprio	Cliente	Opcional; a tela avisa para não escrever dado pessoal; nunca vai para log, view ou relatório (RN08)

Fonte: elaborado pelos autores.

A classificação levou a uma correção que o modelo de dados sozinho não revelaria: a interface de programação da aplicação mostrava a restrição alimentar do cliente também ao Gerente e ao Coordenador, contrariando a RN04, e passou a entregá-la somente ao Garçom e ao Metre, com a comanda aberta. Dois pontos foram deixados para validação com o orientador: o prazo final de retenção da auditoria e a anonimização de nome e e-mail de funcionário desligado, cuja conta não pode ser excluída sem apagar também o histórico que a referencia.

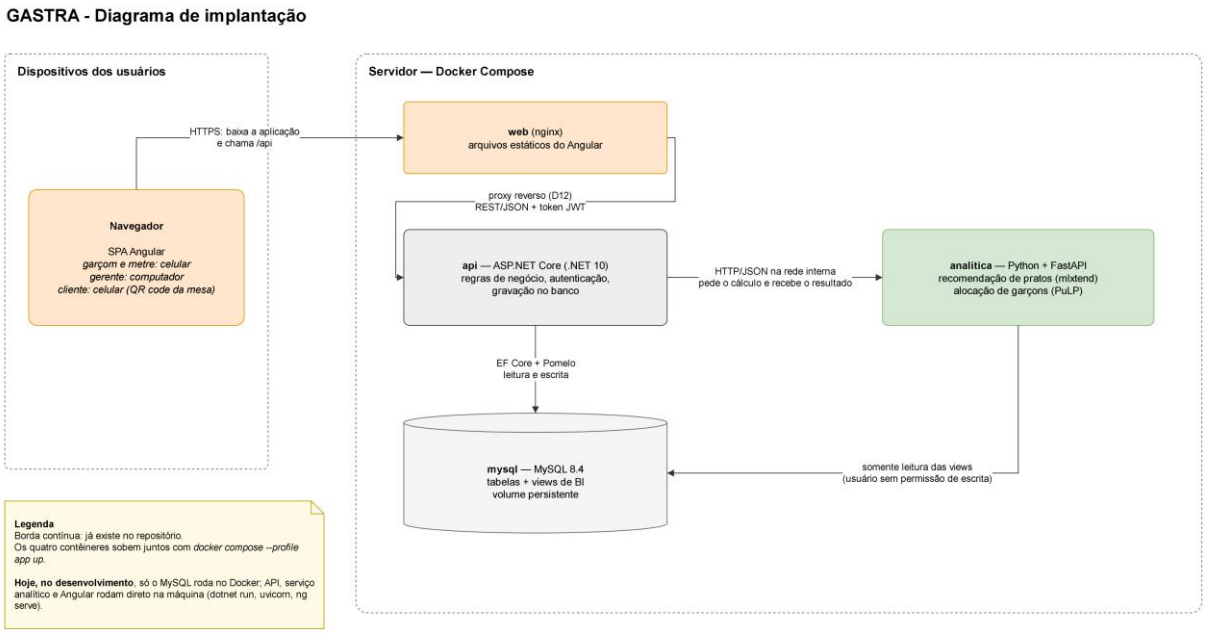
4.4 ARQUITETURA E PROTOTIPAÇÃO (UI/UX)

4.4.1 Visão geral e implantação

O GASTRA é composto por três componentes de software e um banco de dados: o frontend em Angular, executado no navegador; o backend em ASP.NET Core, que concentra as regras de negócio; a camada analítica em Python, que executa os algoritmos de recomendação e de alocação; e o MySQL. O princípio que organiza a arquitetura é que o backend é o único dono das regras de negócio e o único componente que grava no banco: o frontend apenas apresenta e coleta dados, e o serviço analítico apenas calcula e devolve o resultado.

A Figura 7 apresenta o diagrama de implantação. Os quatro componentes são executados em contêineres Docker (DOCKER, 2026) e sobem juntos com um único comando, o que permite executar o sistema completo em qualquer máquina, sem instalar as plataformas de cada um. O contêiner do frontend, com o servidor nginx, entrega os arquivos da aplicação e repassa ao backend as chamadas à interface de programação, de modo que o navegador fala com uma única origem.

Figura 7 – Diagrama de implantação do GASTRA

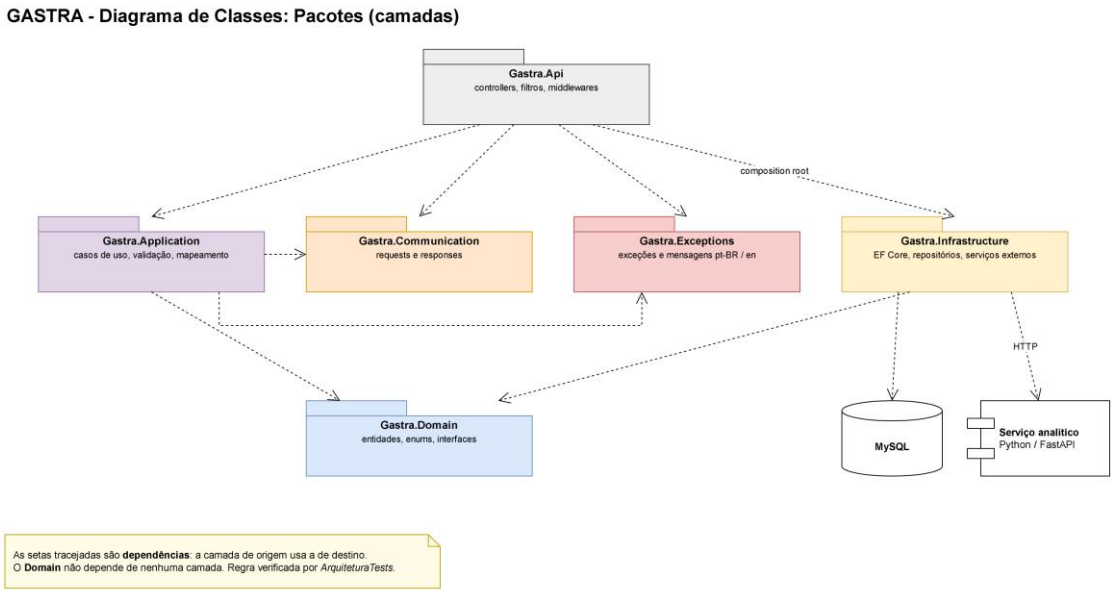


Fonte: elaborado pelos autores.

4.4.2 Backend em camadas

O backend foi organizado em seis projetos, apresentados na Figura 8, cada um com uma responsabilidade: o domínio, com as entidades e as regras de negócio; a aplicação, com um caso de uso por classe; a comunicação, com os contratos de entrada e saída da interface de programação; as exceções e mensagens de erro, traduzidas para português e inglês; a infraestrutura, com o acesso ao banco e ao serviço analítico; e a interface de programação propriamente dita, com os controladores e a configuração. As setas indicam as dependências permitidas: o domínio não depende de nenhuma outra camada, e a infraestrutura implementa interfaces que o domínio define, aplicação do princípio da inversão de dependência descrito na seção 2.2 (SOMMERVILLE, 2019). Essa regra não depende apenas de disciplina: o compilador impede as dependências proibidas, e um teste de arquitetura falha se alguma for introduzida.

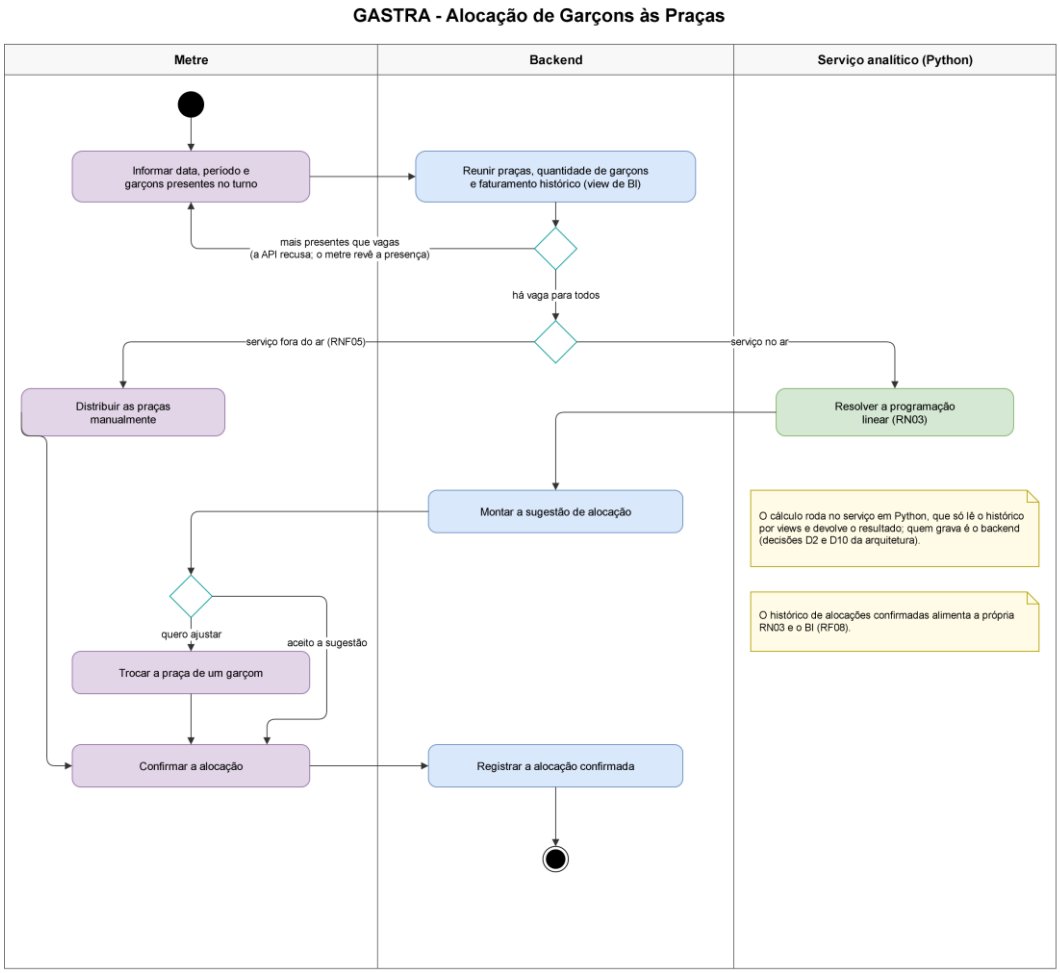
Figura 8 – Camadas do backend e dependências permitidas



Fonte: elaborado pelos autores.

O comportamento dinâmico foi documentado por diagramas de sequência e de atividade. A Figura 9 mostra o fluxo da alocação de garçons, que atravessa as três camadas de software. O Metre informa quem está presente; o backend reúne as praças, as vagas e o histórico de faturamento e recusa o pedido quando há mais presentes do que vagas; com o serviço analítico disponível, a programação linear calcula a sugestão, que o Metre aceita ou ajusta antes de confirmar; com o serviço fora do ar, o Metre distribui as praças manualmente, e o salão não para (RNF05). Um cuidado adicional, que o diagrama não detalha, está no backend: a resposta do serviço é conferida antes de ser gravada, e uma resposta incoerente — que deixe garçom sem praça ou exceda as vagas de uma praça — é tratada como serviço indisponível.

Figura 9 – Diagrama de atividade da alocação de garçons



Fonte: elaborado pelos autores.

4.4.3 Camada analítica

A camada analítica é um serviço HTTP em FastAPI (RAMÍREZ, 2026). A recomendação de pratos (RF09) combina duas técnicas de ciência de dados. A primeira são as regras de associação, obtidas pelo algoritmo Apriori (AGRAWAL; SRIKANT, 1994), da biblioteca mlxtend (RASCHKA, 2018), sobre o histórico de itens pedidos juntos em cada comanda: uma regra "quem pede A costuma pedir B" só é usada com suporte mínimo de 5%, confiança mínima de 30% e lift acima de 1, porque um lift abaixo de 1 indica itens que se evitam, e sugeri-los seria pior do que não sugerir nada. O histórico é lido das views do banco por um usuário que não tem permissão de escrita nem acesso às tabelas; enquanto houver menos de 50 comandas fechadas, o serviço usa um histórico simulado e informa esse motivo na resposta.

A segunda técnica é a clusterização. Cada comanda do histórico é representada por um vetor com o valor 1 nos itens que ela levou e 0 nos demais, normalizado pelo próprio tamanho, de modo que a distância entre duas comandas passa a medir a proporção do consumo que elas compartilham (similaridade do cosseno), e não a quantidade de itens. O algoritmo K-Means (MACQUEEN, 1967), da biblioteca scikit-learn (PEDREGOSA et al., 2011), agrupa as comandas parecidas em perfis de consumo. Como o K-Means precisa saber quantos grupos procurar, testam-se de dois a seis, e escolhe-se a quantidade com maior coeficiente de silhueta (ROUSSEEUW, 1987), medida que compara, para cada comanda, a distância ao próprio grupo com a distância ao grupo vizinho; abaixo de 0,25, considera-se que não há perfis de fato. Nenhum atributo do cliente entra no cálculo (RN05): os perfis são definidos apenas pelo que foi pedido.

As duas técnicas trabalham juntas na recomendação segmentada. A comanda em andamento é associada ao perfil mais parecido com o que ela já tem e recebe sugestões de três fontes: as regras de associação apuradas somente entre as comandas do perfil; os itens que caracterizam o perfil e que a mesa ainda não pediu; e, para completar, as regras gerais do restaurante. As duas primeiras estimam a mesma coisa — a chance de a mesa pedir o item — e por isso são ordenadas juntas, pela confiança, de modo que uma regra fraca não passe à frente de um item que quase toda mesa do perfil pede. Quando a clusterização não encontra perfis, a recomendação usa apenas as regras gerais. Os perfis encontrados também são apresentados ao Gerente, nos relatórios de Business Intelligence.

A alocação de garçons (RF06) foi modelada como um problema de designação, caso particular do problema de transporte em programação linear (HILLIER; LIEBERMAN, 2006), e resolvida com a biblioteca PuLP (MITCHELL; O'SULLIVAN; DUNNING, 2011). A variável de decisão indica se um garçom é alocado a uma praça; a função objetivo minimiza a soma dos custos das designações, em que o custo combina, com os pesos w1 e w2 da RN03, o desequilíbrio de faturamento e o tempo desde a última praça de alto potencial, ambos normalizados entre zero e um; as restrições garantem que cada garçom receba exatamente uma praça e que nenhuma praça exceda as suas vagas. Por ser um problema de designação, a matriz de restrições é totalmente unimodular, e a solução da relaxação contínua já é inteira. A calibração dos pesos é apresentada no capítulo 5.

4.4.4 Decisões de arquitetura

As decisões de arquitetura foram registradas à medida que foram tomadas, cada uma com a alternativa considerada e o custo aceito, para que a escolha pudesse ser revista com o seu contexto. O Quadro 7 resume as principais.

Quadro 7 – Principais decisões de arquitetura

#	**Decisão**	**Alternativa considerada**	**Motivo**
D1	Backend em camadas separadas em projetos	Um único projeto com pastas	O compilador impede dependências proibidas, e um teste verifica a regra
D2	Algoritmos em um serviço Python separado, chamado por HTTP	Implementar tudo em C#	Bibliotecas maduras de regras de associação e programação linear
D3	Núcleo de comandas independente do serviço analítico	Consultar o Python ao abrir mesa ou lançar item	RNF05: sem o Python, só as sugestões ficam indisponíveis
D5	Banco gerado por migrations (Code First)	Escrever o esquema em SQL	O esquema é versionado e revisado junto do código
D6	Regras de negócio no domínio, sem stored procedures	Procedures para fechamento e taxa	A regra fica em um só lugar e é testada sem banco
D7	Autenticação por JWT (JONES; BRADLEY; SAKIMURA, 2015) com chave de sessão	Sessão em cookie no servidor	A chave de sessão permite invalidar o token no logoff e na inativação
D10	Serviço analítico lê o histórico só por views, com usuário somente leitura	Enviar todo o histórico a cada chamada	A view entrega só colunas agregadas, sem dado pessoal
D11	Dados simulados gerados por ferramenta que grava pelo domínio	Script SQL direto	Os dados respeitam as mesmas regras do sistema real
D12	nginx serve o frontend e repassa as chamadas à API	Origens separadas com CORS	Uma única origem; o celular na rede local acessa sem recompilar
D13	Bloqueio de conta e limite de requisições por IP	Apenas uma das duas proteções	Cada uma cobre o ataque que a outra não vê
D14	Explicação da alocação ao Metre por faixas, e não por valores	Mostrar o faturamento de cada garçom	O Metre não consulta indicador de desempenho

Fonte: elaborado pelos autores.

5 DESENVOLVIMENTO, RESULTADOS E TESTES

6 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

AGRAWAL, Rakesh; SRIKANT, Ramakrishnan. Fast algorithms for mining association rules. In: INTERNATIONAL CONFERENCE ON VERY LARGE DATA BASES, 20., 1994, Santiago. Proceedings [...]. San Francisco: Morgan Kaufmann, 1994. p. 487-499.

ASSOCIAÇÃO BRASILEIRA DE BARES E RESTAURANTES (ABRASEL). Bares e restaurantes faturaram mais em 2025, diz pesquisa de serviços. [S. l.], 2025a. Disponível em: https://abrasel.com.br/noticias/noticias/bares-e-restaurantes-faturaram-mais-em-2025-diz-pesquisa-de-servicos/. Acesso em: 15 jun. 2026.

ASSOCIAÇÃO BRASILEIRA DE BARES E RESTAURANTES (ABRASEL). Salário em bares e restaurantes segue crescendo, mas contratação ainda é desafio. [S. l.], 2025b. Disponível em: https://abrasel.com.br/noticias/noticias/salario-em-bares-e-restaurantes-segue-crescendo-mas-contratacao-ainda-e-desafio/. Acesso em: 15 jun. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 15 jun. 2026.

CAVIQUE, Luís. Micro-segmentação de clientes com base em dados de consumo: modelo RM-Similis. Revista Portuguesa e Brasileira de Gestão, Rio de Janeiro, v. 2, n. 3, p. 72-77, 2003. Disponível em: https://periodicos.fgv.br/rbpg/article/view/79326. Acesso em: 15 jun. 2026.

COELHO, Michella Aguiar. Programação matemática aplicada a casos reais de designação de entregadores em uma empresa de food delivery. 2021. Dissertação (Mestrado em Engenharia Elétrica e Informática Industrial) — Universidade Tecnológica Federal do Paraná, Curitiba, 2021. Disponível em: https://repositorio.utfpr.edu.br/jspui/handle/1/27086. Acesso em: 15 jun. 2026.

DOCKER INC. Docker Docs. [S. l.], 2026. Disponível em: https://docs.docker.com/. Acesso em: 30 out. 2026.

DRIESSEN, Vincent. A successful Git branching model. [S. l.], 2010. Disponível em: https://nvie.com/posts/a-successful-git-branching-model/. Acesso em: 30 out. 2026.

GINI, Corrado. Variabilità e mutabilità: contributo allo studio delle distribuzioni e delle relazioni statistiche. Bologna: Tipografia di Paolo Cuppini, 1912.

GITHUB. GitHub Actions documentation. [S. l.], 2026. Disponível em: https://docs.github.com/actions. Acesso em: 30 out. 2026.

GONÇALVES, Julio Cesar Leite; SOLER, Luciano. Extraindo conhecimento com análise das cestas de compras através do Data Mining. In: ENCONTRO DE ATIVIDADES CIENTÍFICAS, 15., 2012. Anais [...]. [S. l.]: Unopar, 2012. Disponível em: https://repositorio.pgsscogna.com.br/handle/123456789/2377. Acesso em: 15 jun. 2026.

GOOGLE. Angular. [S. l.], 2026. Disponível em: https://angular.dev/. Acesso em: 30 out. 2026.

HILLIER, Frederick S.; LIEBERMAN, Gerald J. Introdução à pesquisa operacional. Tradução Ariovaldo Griesi; revisão técnica João Chang Junior. 8. ed. São Paulo: McGraw-Hill, 2006. Disponível em: https://www.academia.edu/29281242/HILLIER_Frederick_S_LIEBERMAN_Gerald_J_Introdução_à_Pesquisa_Operacional. Acesso em: 19 set. 2026.

HUALLPA TAPIA, Luis David et al. Application of Business Intelligence to Improve Utilities by Increasing Customer Satisfaction in Restaurants. In: LACCEI INTERNATIONAL MULTI-CONFERENCE FOR ENGINEERING, EDUCATION, AND TECHNOLOGY, 17., 2019, Montego Bay. Anais [...]. Montego Bay: LACCEI, 2019. Disponível em: https://www.laccei.org/LACCEI2019-MontegoBay/full_papers/FP111.pdf. Acesso em: 15 jun. 2026.

JONES, Michael B.; BRADLEY, John; SAKIMURA, Nat. JSON Web Token (JWT). RFC 7519. [S. l.]: Internet Engineering Task Force, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: 30 out. 2026.

KUBERNS. Azure Pricing 2026: Real Costs, Plans & Cheaper Options. [S. l.], 2026. Disponível em: https://kuberns.com/blogs/azure-pricing/. Acesso em: 19 set. 2026.

MACQUEEN, James B. Some methods for classification and analysis of multivariate observations. In: BERKELEY SYMPOSIUM ON MATHEMATICAL STATISTICS AND PROBABILITY, 5., 1967, Berkeley. Proceedings [...]. Berkeley: University of California Press, 1967. v. 1, p. 281-297.

MARTÍNEZ ORTIZ, Rafael Humberto. Condiciones óptimas para implementar Business Intelligence en la productividad de MIPYMES: caso de estudio en restaurantes. 2025. Trabajo Final de Graduación (Máster en Administración de Proyectos) — Universidad Tecnológica Centroamericana (UNITEC), Tegucigalpa, 2025. Disponível em: https://repositorio.unitec.edu/server/api/core/bitstreams/c913cbac-c99d-4da7-bfc7-92cd733ae601/content. Acesso em: 15 jun. 2026.

MICROSOFT. ASP.NET Core documentation. [S. l.], 2026a. Disponível em: https://learn.microsoft.com/aspnet/core/. Acesso em: 30 out. 2026.

MICROSOFT. Entity Framework Core. [S. l.], 2026b. Disponível em: https://learn.microsoft.com/ef/core/. Acesso em: 30 out. 2026.

MITCHELL, Stuart; O'SULLIVAN, Michael; DUNNING, Iain. PuLP: a linear programming toolkit for Python. Auckland: The University of Auckland, 2011. Disponível em: https://optimization-online.org/2011/09/3178/. Acesso em: 30 out. 2026.

MOREIRA, Eduardo Henrique Fonseca. Desenvolvimento de sistema de feedback de consumo integrado a software de restaurante. 2018. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Software) — Universidade de Brasília, Brasília, 2018. Disponível em: https://bdm.unb.br/bitstream/10483/21558/1/2018_EduardoHenriqueFonsecaMoreira_tcc.pdf. Acesso em: 18 set. 2026.

NIELSEN, Jakob. Heuristic evaluation. In: NIELSEN, Jakob; MACK, Robert L. (ed.). Usability inspection methods. New York: John Wiley & Sons, 1994. p. 25-62.

NIELSEN, Jakob. Response Time: The 3 Important Limits. Nielsen Norman Group, 1993. Disponível em: https://www.nngroup.com/articles/response-times-3-important-limits/. Acesso em: 19 set. 2026.

ORACLE. MySQL 8.4 reference manual. [S. l.], 2026. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/. Acesso em: 30 out. 2026.

OWASP FOUNDATION. OWASP Top 10:2021. [S. l.], 2021. Disponível em: https://owasp.org/Top10/2021/. Acesso em: 30 out. 2026.

PEDREGOSA, Fabian et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research, v. 12, p. 2825-2830, 2011.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. Tradução Ariovaldo Griesi, Mario Moro Fecchio; revisão técnica Reginaldo Arakaki, Julio Arakaki, Renato Manzan de Andrade. 7. ed. Porto Alegre: AMGH, 2011. Disponível em: https://www.kufunda.net/publicdocs/Engenharia%20de%20Software%20-%207.ed.%20(Roger%20S.%20Pressman).pdf. Acesso em: 19 set. 2026.

PUMP.CO. Azure Database for MySQL: Features, Pricing & Benefits. [S. l.], 2025. Disponível em: https://www.pump.co/blog/azure-database-for-mysql/. Acesso em: 19 set. 2026.

RAMÍREZ, Sebastián. FastAPI. [S. l.], 2026. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 30 out. 2026.

RAPÔSO, Cláudio Filipe Lima et al. LGPD – Lei Geral de Proteção de Dados Pessoais em Tecnologia da Informação: revisão sistemática. RACE – Revista de Administração do Cesmac, v. 4, p. 58-67, 2019. Disponível em: https://cesmac.emnuvens.com.br/administracao/article/view/1035. Acesso em: 15 jun. 2026.

RASCHKA, Sebastian. MLxtend: providing machine learning and data science utilities and extensions to Python's scientific computing stack. Journal of Open Source Software, v. 3, n. 24, p. 638, 2018. DOI: 10.21105/joss.00638.

REGISTRO.BR. Sobre domínios. [S. l.], 2026. Disponível em: https://registro.br/dominio/. Acesso em: 19 set. 2026.

ROUSSEEUW, Peter J. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of Computational and Applied Mathematics, v. 20, p. 53-65, 1987. DOI: 10.1016/0377-0427(87)90125-7.

SAIPOS. Sistema de gestão da Saipos: saiba como funciona. [S. l.], [20--]. Disponível em: https://saipos.com/como-funciona. Acesso em: 18 set. 2026.

SANTOS, Vitor Luis dos; GIBERTONI, Daniela. Os impactos do Business Intelligence para tomada de decisões. Interface Tecnológica, Taquaritinga, v. 19, n. 2, p. 258-269, 2022. Disponível em: https://revista.fatectq.edu.br/interfacetecnologica/pt_BR/article/view/1524/807. Acesso em: 15 jun. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. O Guia do Scrum: o guia definitivo para o Scrum: as regras do jogo. [S. l.], 2020. Disponível em: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-PortugueseBR-3.0.pdf. Acesso em: 30 out. 2026.

SOMMERVILLE, Ian. Engenharia de software. Tradução Fábio Levy Siqueira. 10. ed. São Paulo: Pearson Education do Brasil, 2019. Disponível em: https://archive.org/details/sommerville-engenharia-de-software-10e/page/n5/mode/2up. Acesso em: 19 set. 2026.

STRATEGYZER. Business Model Canvas. [S. l.], [20--]. Disponível em: https://www.strategyzer.com/canvas/business-model-canvas. Acesso em: 18 set. 2026.

SYED MARZUKI, Sharifah Zannierah; HALL, Colin Michael; BALLANTINE, Paul William. Measurement of Restaurant Manager Expectations toward Halal Certification Using Factor and Cluster Analysis. Procedia – Social and Behavioral Sciences, v. 121, p. 291-303, 2014. Disponível em: https://doi.org/10.1016/j.sbspro.2014.01.1130. Acesso em: 15 jun. 2026.

VIEIRA, Carolina Luísa dos Santos et al. Determinação de escalas de trabalho de funcionários de restaurantes considerando preferências. Produção em Foco, p. 79-99, 2015. Disponível em: https://www.researchgate.net/publication/299413755. Acesso em: 15 jun. 2026.

ZANIOL, Cristina. Alocação de funcionários: um problema de programação linear em emparelhamento de grafos bipartidos. In: SALÃO DE INICIAÇÃO CIENTÍFICA, 23., 2011, Porto Alegre. Resumos [...]. Porto Alegre: UFRGS, 2011. Disponível em: http://hdl.handle.net/10183/48559. Acesso em: 15 jun. 2026.